UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

# CS 498: Machine Learning System Spring 2026

Minjia Zhang

The Grainger College of Engineering

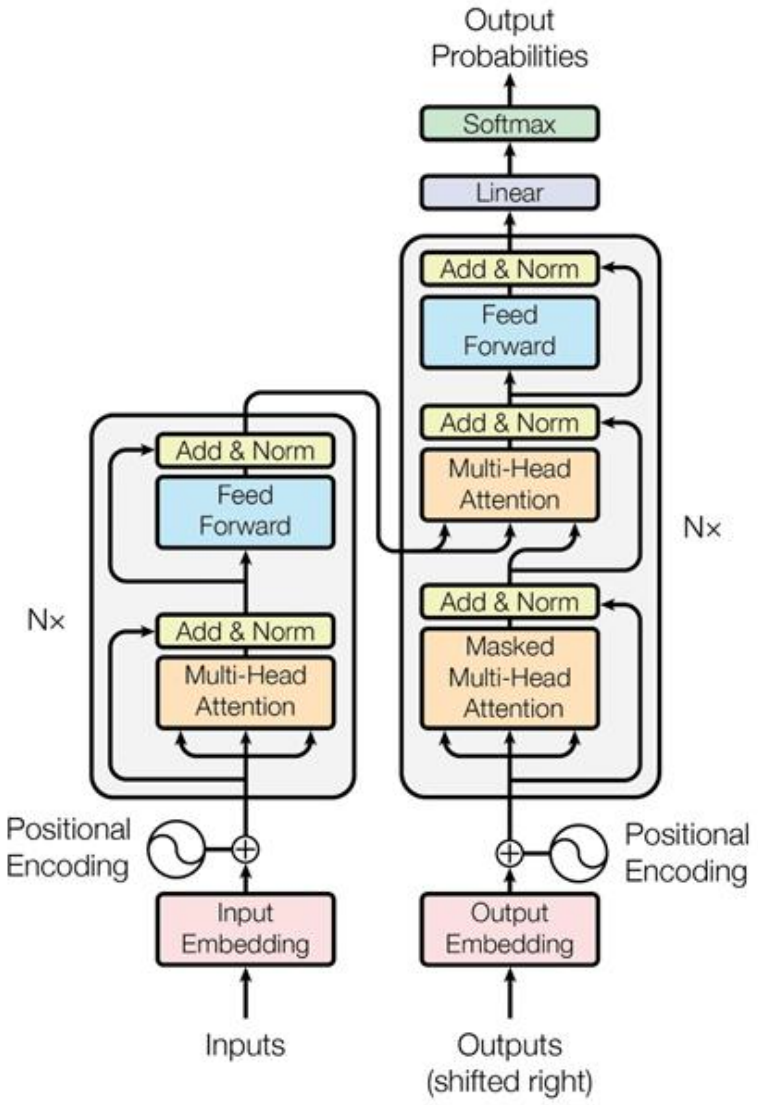
# Today

- **Transformers Deep Dive**
- **Learning Objectives**
  - Explain the core computation and dataflow of Transformers

# Transformers

- Tokenization
- Input embeddings
- Position encodings
- Query, Key, Value
- Attention
- Self-attention
- Multi-head attention
- Feed forward
- Add & norm
- Residual

- Masked attention
- Causal attention
- Linear
- Softmax
- Encoders
- Decoder
- Encoder-decoder



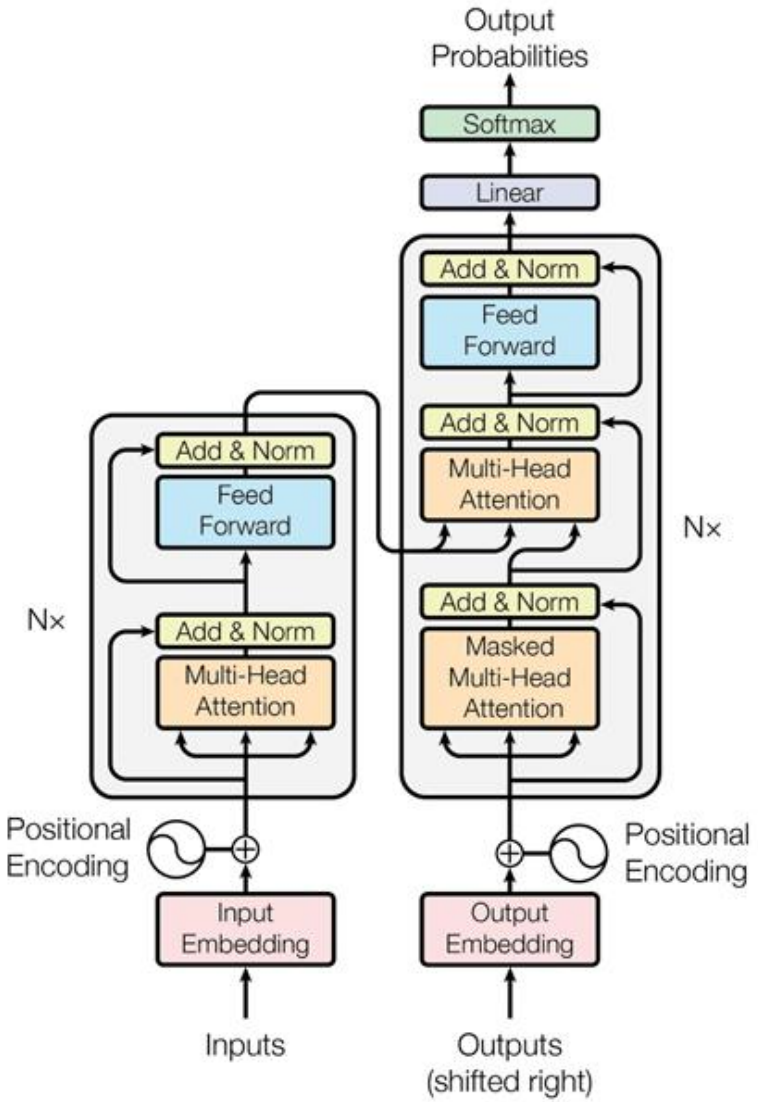
# Machine Translation

Targets

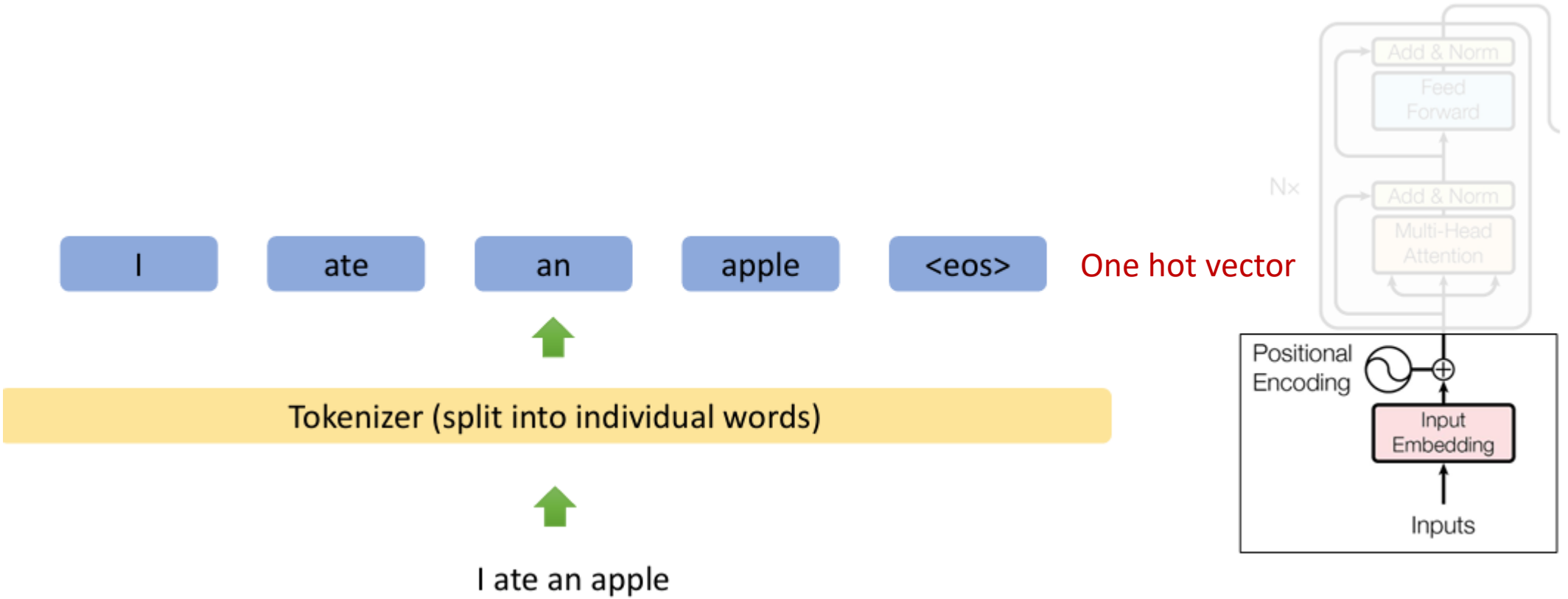
Ich habe einen Apfel gegessen

Inputs

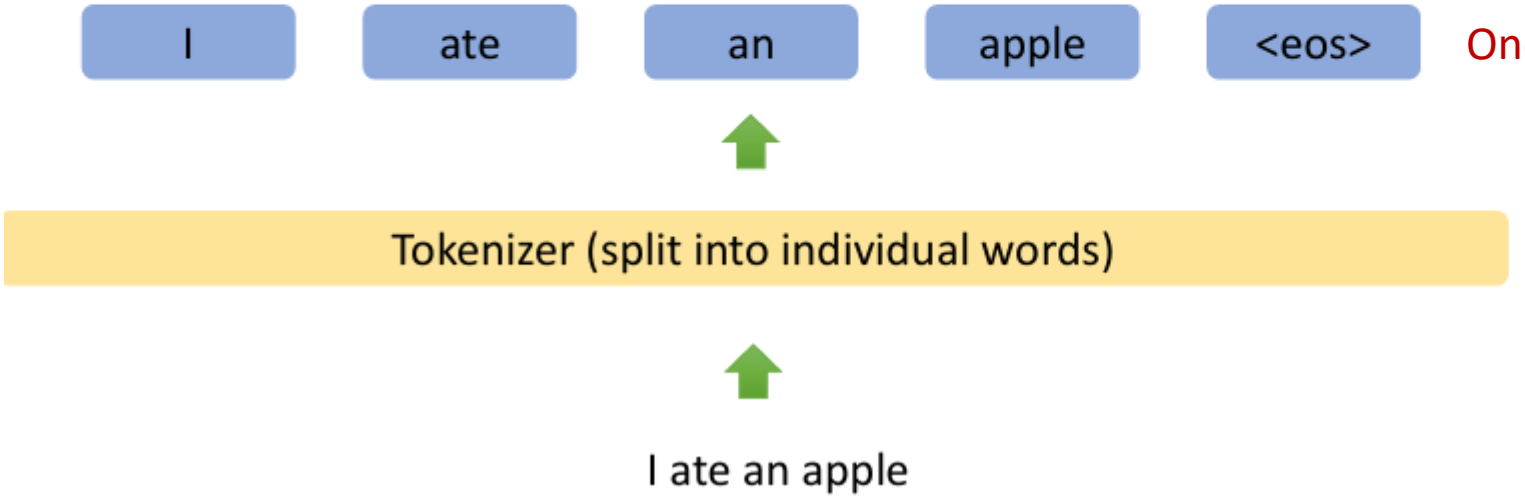
I ate an apple



# Tokenization



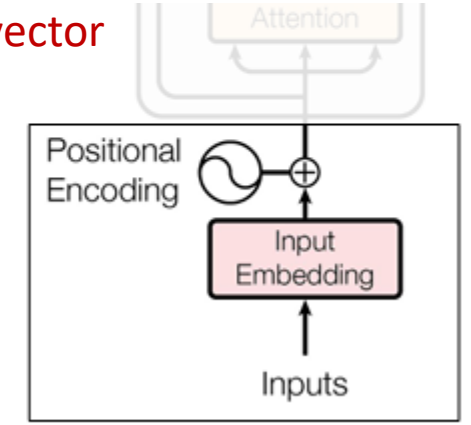
# Tokenization



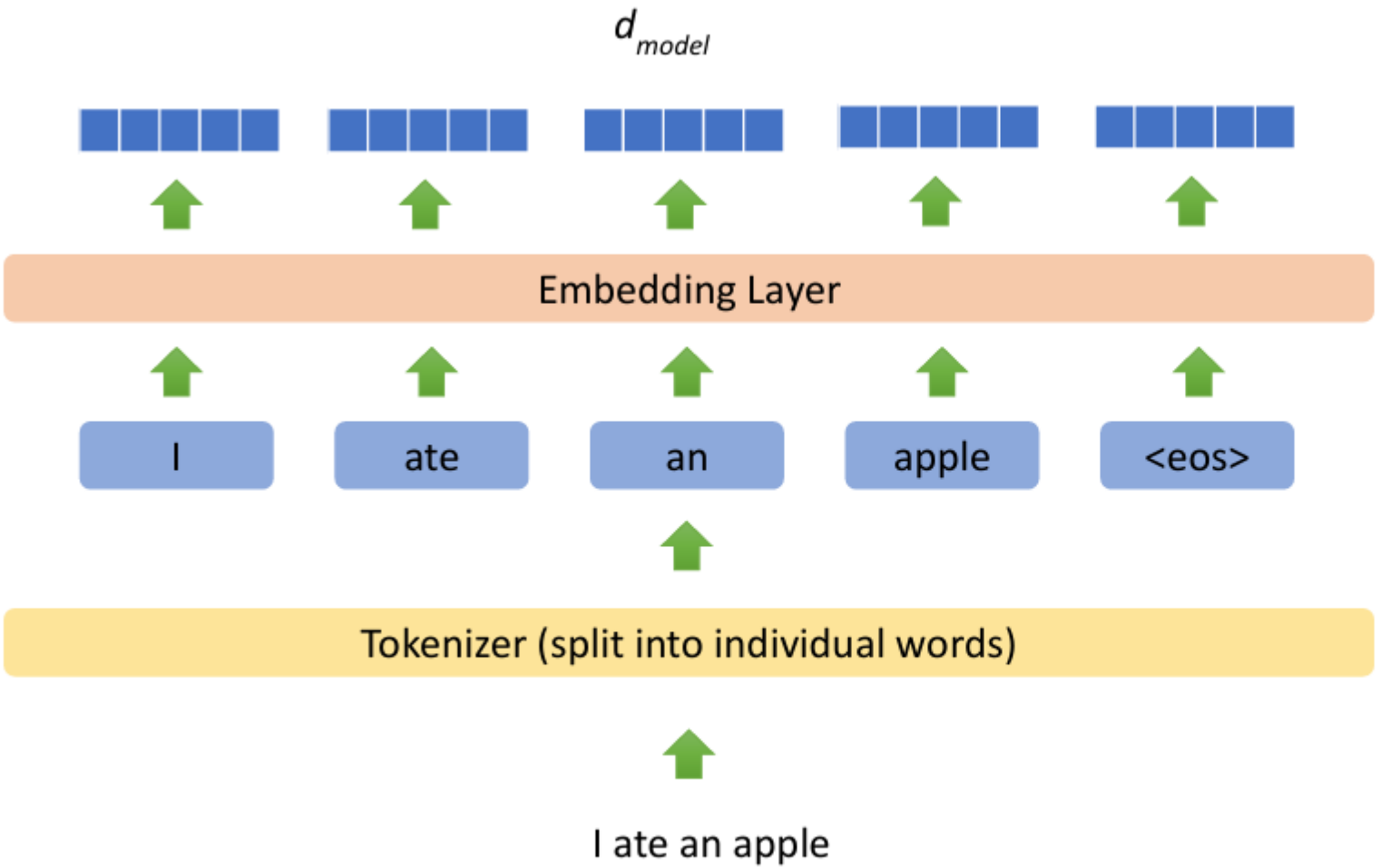

**One-hot encoding**

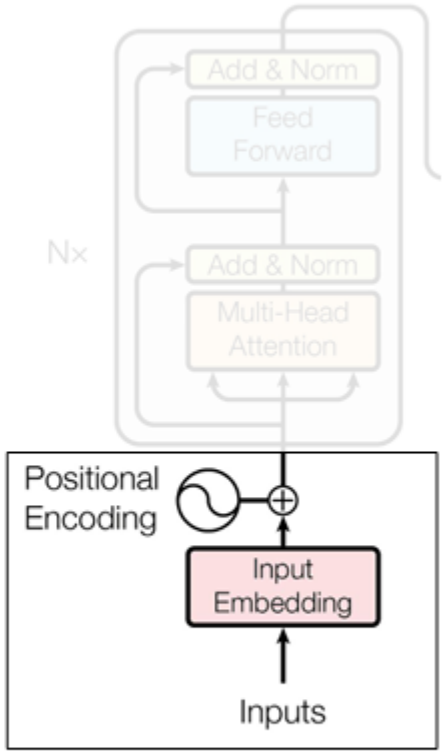
| | cat | mat | on | sat | the |
|---|---|---|---|---|---|
| **the** => | 0 | 0 | 0 | 0 | 1 |
| **cat** => | 1 | 0 | 0 | 0 | 0 |
| **sat** => | 0 | 0 | 0 | 1 | 0 |
| ... | | | | | |

One hot vector



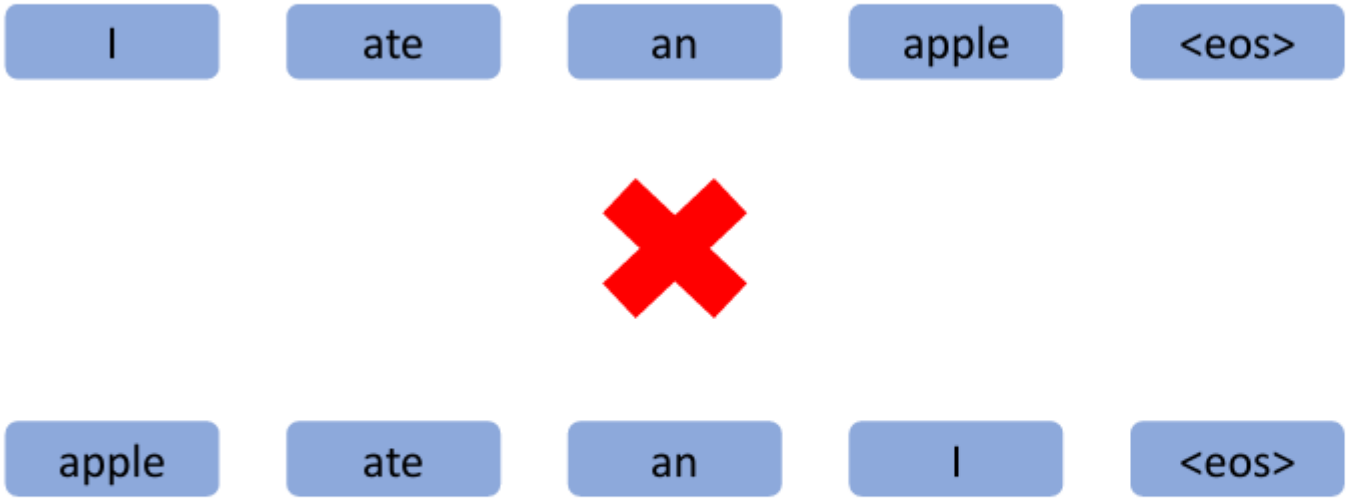
# Input Embeddings

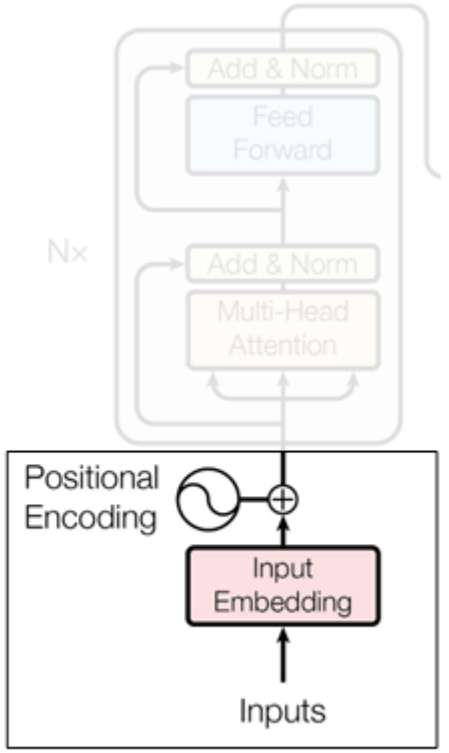



Generate Input Embeddings

# Position Matters

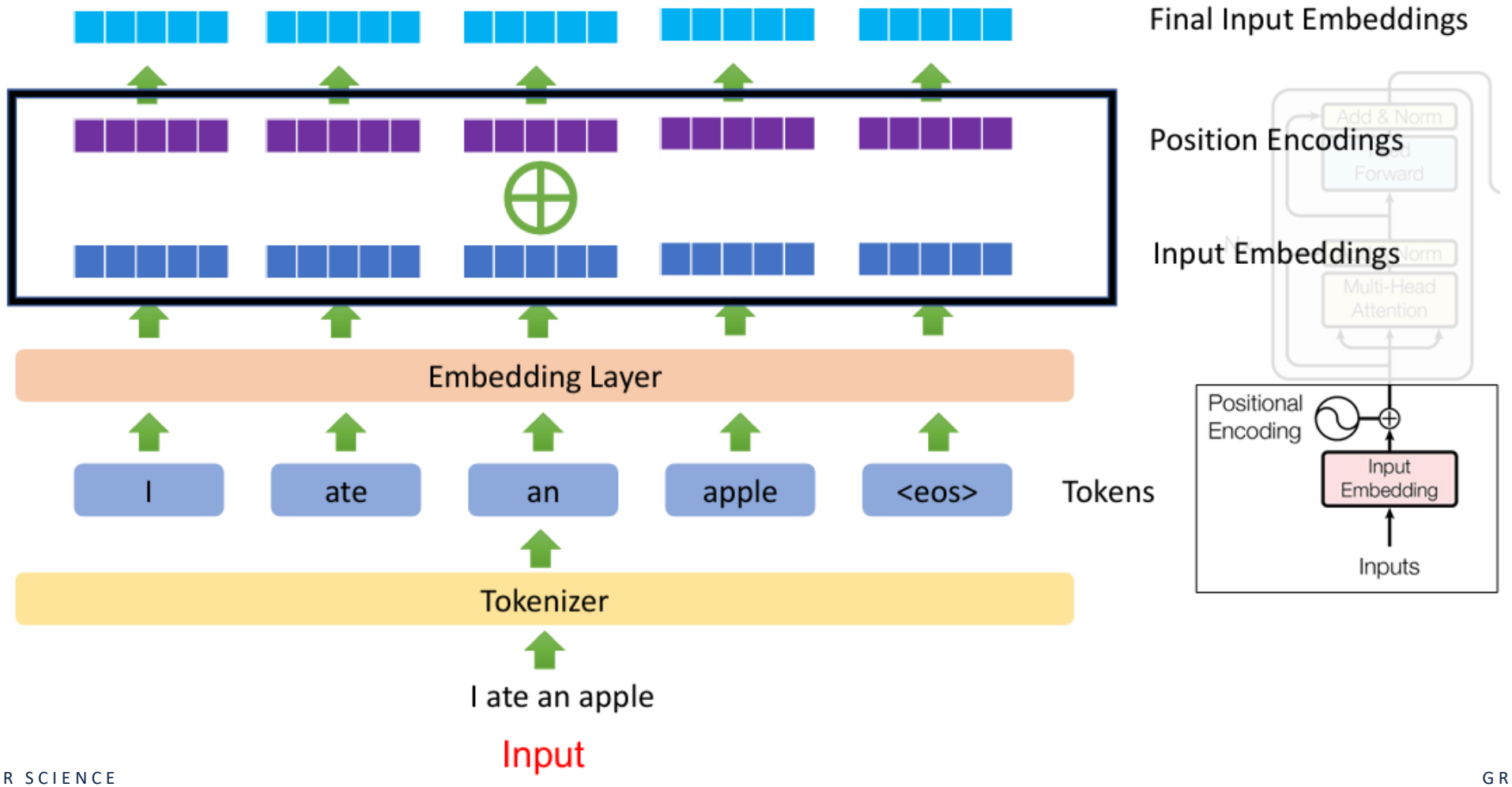



- The position of words in a sentence carries information!
- **Idea:** Add some information to the representation at the beginning that indicates where it is in the sentence

# Position Encodings

Absolute positional encoding (original Transformer): The token embeddings and the absolute position embedding are added together element-wise.



# Position Encodings

- The original position embedding in Transformer is not ideal, because absolute position is less important than relative position

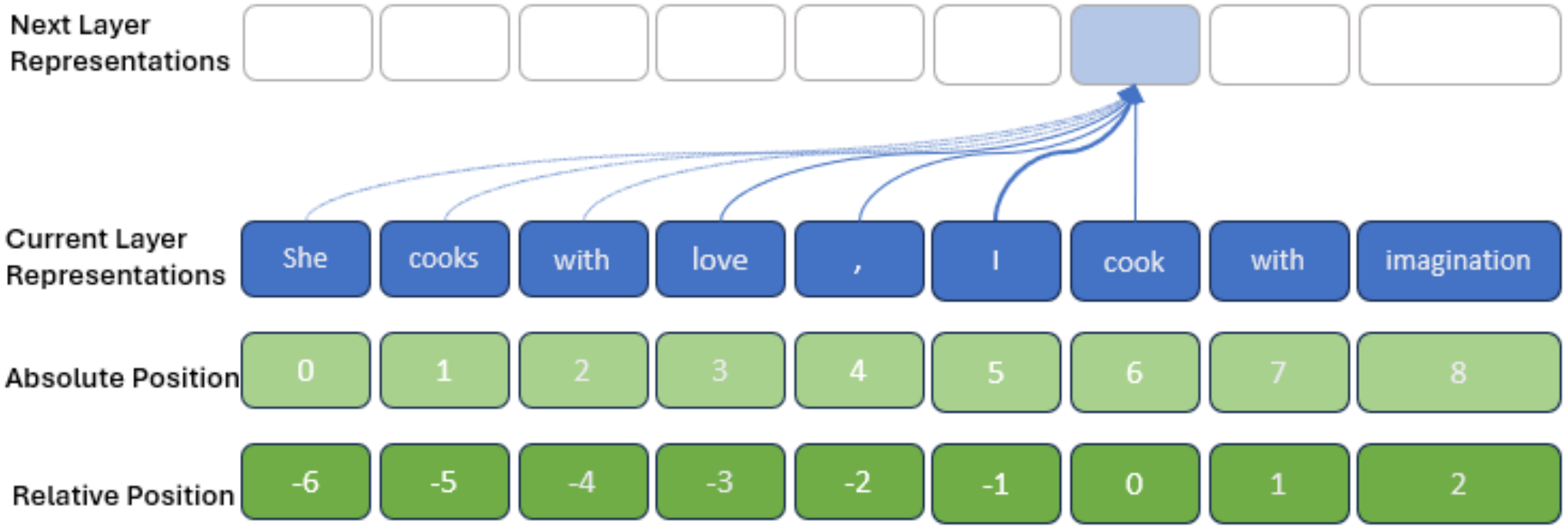
I walk my dog every day

every single day I walk my dog

The fact that “my dog” is right after “I walk” is the important part, not its absolute position

# Relative Position Encodings

- Translation invariance to position information
- Lead to performance improvements



# Rotary Position Embedding

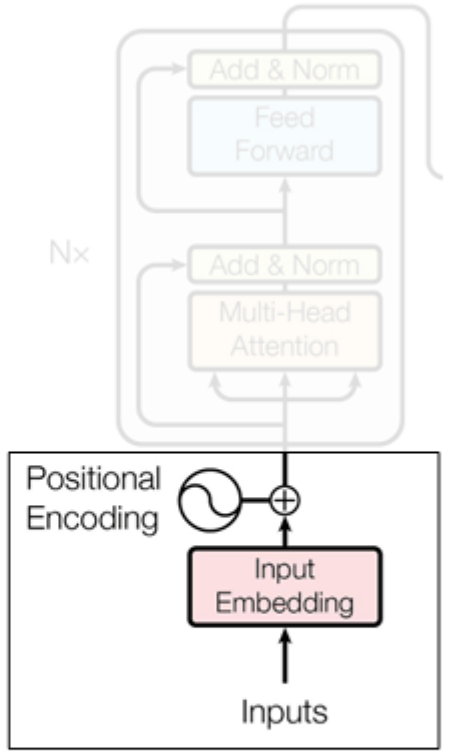
- Most recent LLMs adopt RoPE

ROFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W^{(11)}_{\{q,k\}} & W^{(12)}_{\{q,k\}} \\ W^{(21)}_{\{q,k\}} & W^{(22)}_{\{q,k\}} \end{pmatrix} \begin{pmatrix} x^{(1)}_m \\ x^{(2)}_m \end{pmatrix}$$

Table 2: Comparing RoFormer and BERT by fine tuning on downstream GLEU tasks.

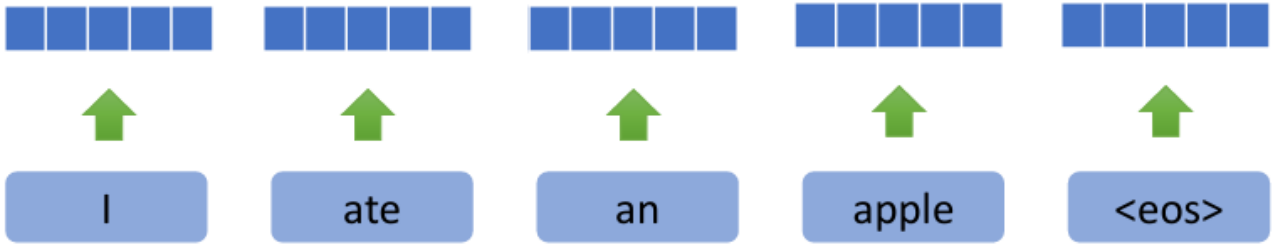
| Model | MRPC | SST-2 | QNLI | STS-B | QQP | MNLI(m/mm) |
|---|---|---|---|---|---|---|
| BERTDevlin et al. [2019] | 88.9 | 93.5 | 90.5 | 85.8 | 71.2 | 84.6/83.4 |
| RoFormer | **89.5** | 90.7 | 88.0 | **87.0** | **86.4** | 80.2/79.8 |

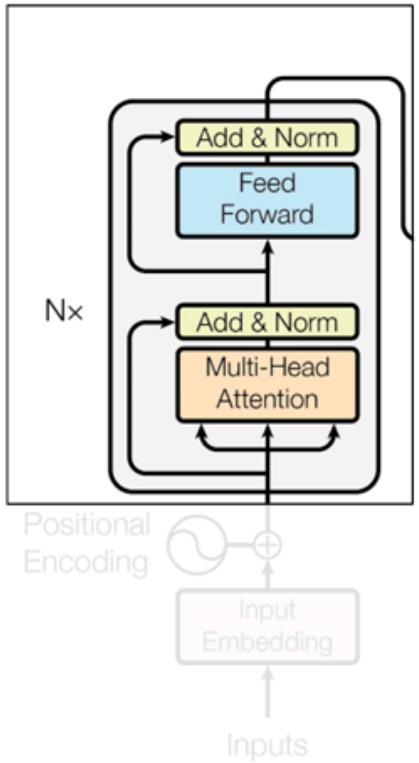



REF: Rotary Position Embeddings

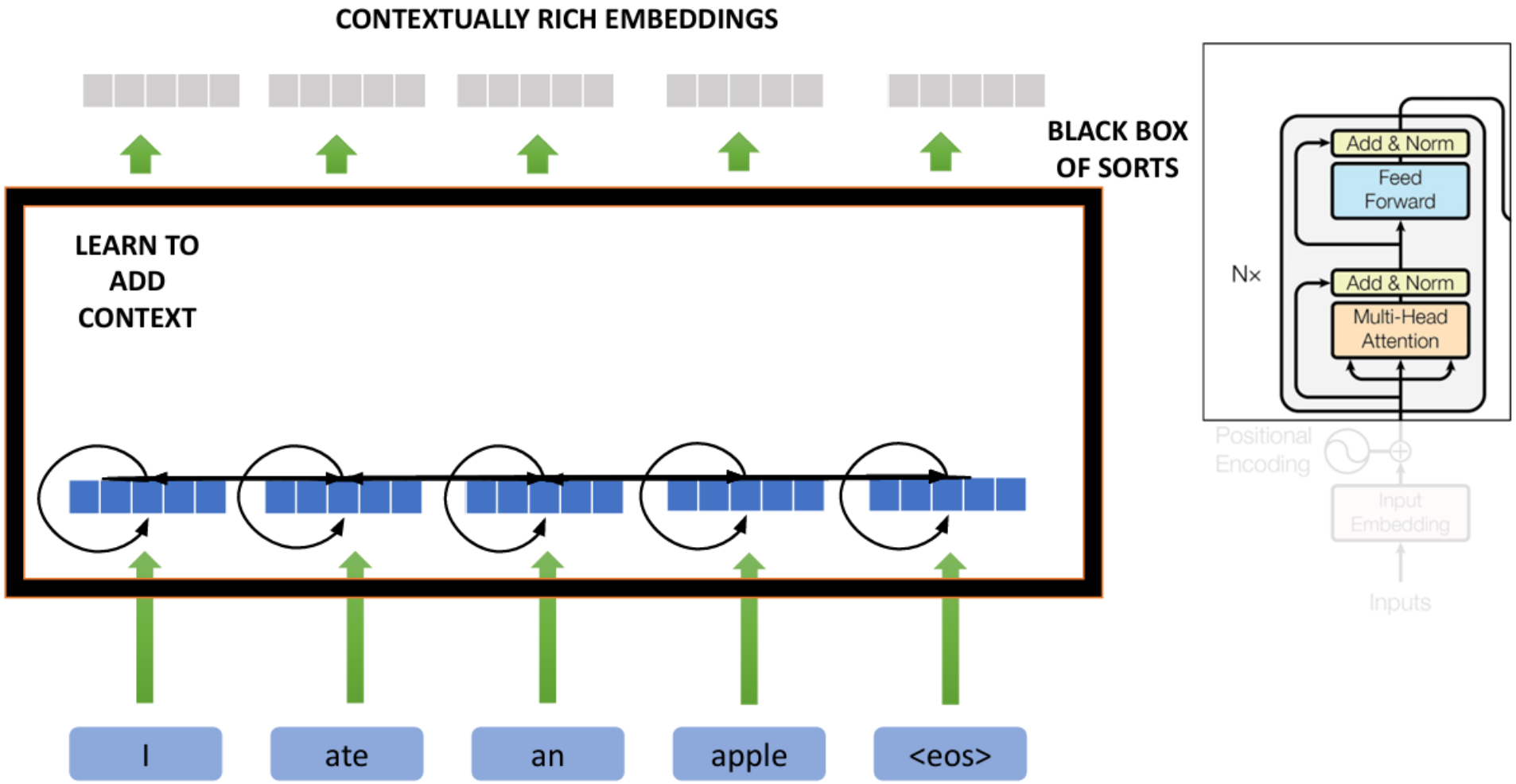
# Encoder

WHERE IS THE CONTEXT ?

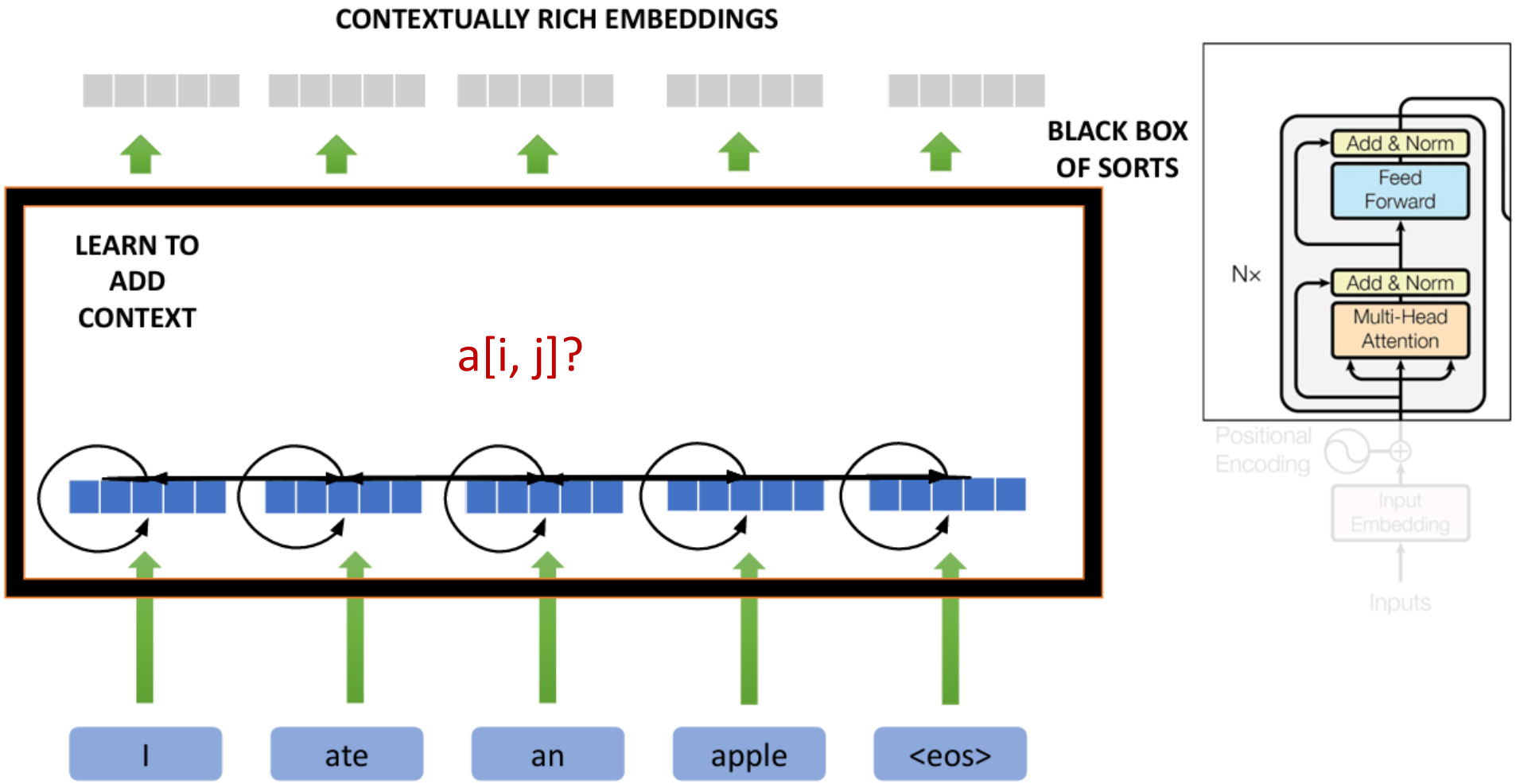


# Encoder


CONTEXTUALLY RICH EMBEDDINGS
BLACK BOX OF SORTS
LEARN TO ADD CONTEXT
I
ate
an
apple
<eos>
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Inputs

# Encoder

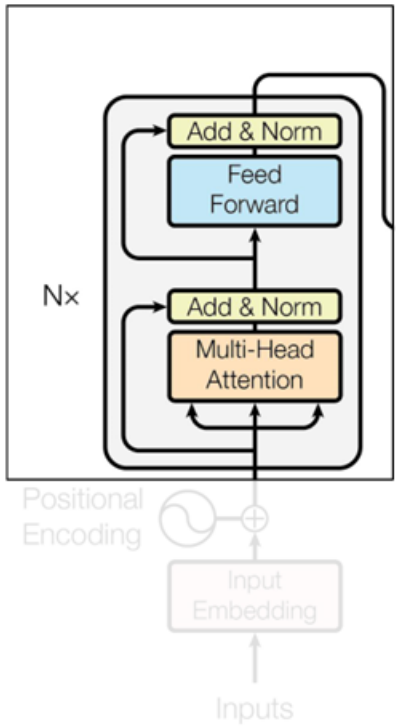

CONTEXTUALLY RICH EMBEDDINGS
BLACK BOX OF SORTS
LEARN TO ADD CONTEXT
a[i, j]?
I
ate
an
apple
<eos>
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Inputs

# Multi-Head Self-Attention

- Before “multi-head” attention, what is “single head” attention?

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

- Query
- Key
- Value



# Multi-Head Self-Attention: Query, Key, & Value

- Database {key, value store}

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}


```
{"order_100": {"items":"a1", "delivery_date":"a2", ...}},
{"order_101": {"items":"b1", "delivery_date":"b2", ...}},
{"order_102": {"items":"c1", "delivery_date":"c2", ...}},
{"order_103": {"items":"d1", "delivery_date":"d2", ...}},
{"order_104": {"items":"e1", "delivery_date":"e2", ...}},
{"order_105": {"items":"f1", "delivery_date":"f2", ...}},
{"order_106": {"items":"g1", "delivery_date":"g2", ...}},
{"order_107": {"items":"h1", "delivery_date":"h2", ...}},
{"order_108": {"items":"i1", "delivery_date":"i2", ...}},
{"order_109": {"items":"j1", "delivery_date":"j2", ...}},
{"order_110": {"items":"k1", "delivery_date":"k2", ...}}
```


**Query**

1. Search for info

**Key**

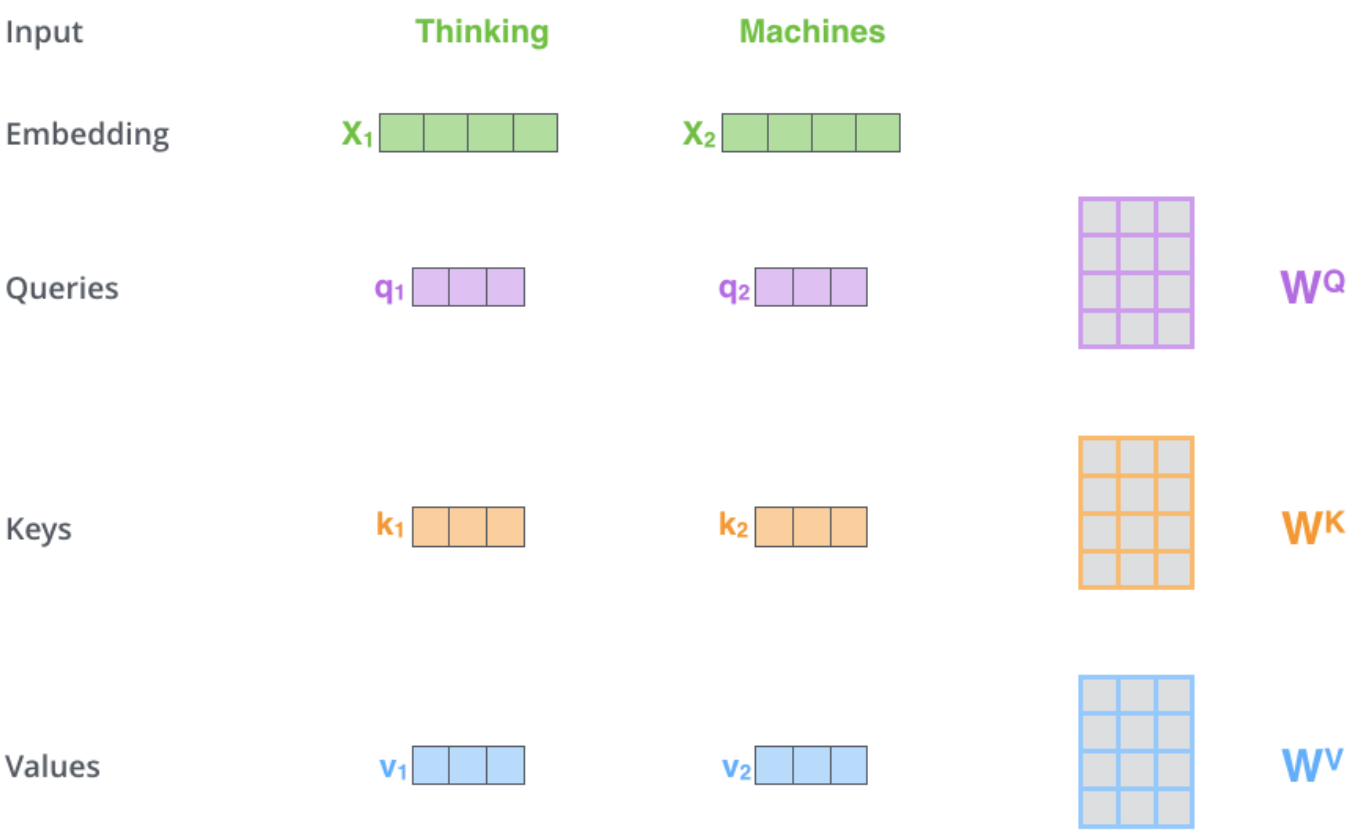
1. Interacts directly with Queries
2. Distinguishes one object from another
3. Identify which object is the most relevant and by how much

**Value**

1. Actual details of the object
2. More fine grained

# Self-Attention

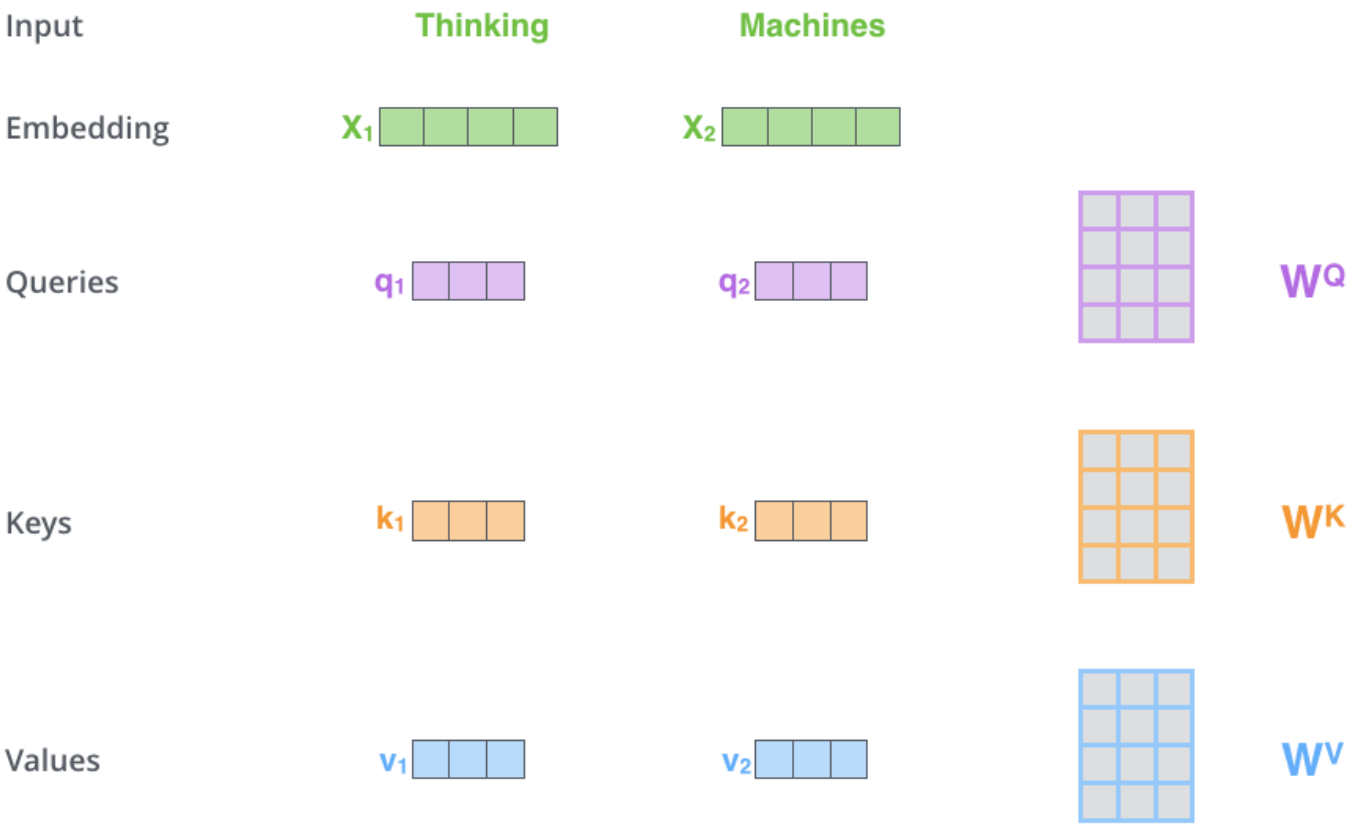
- Step 1: compute “key, value, query” embedding for each input token

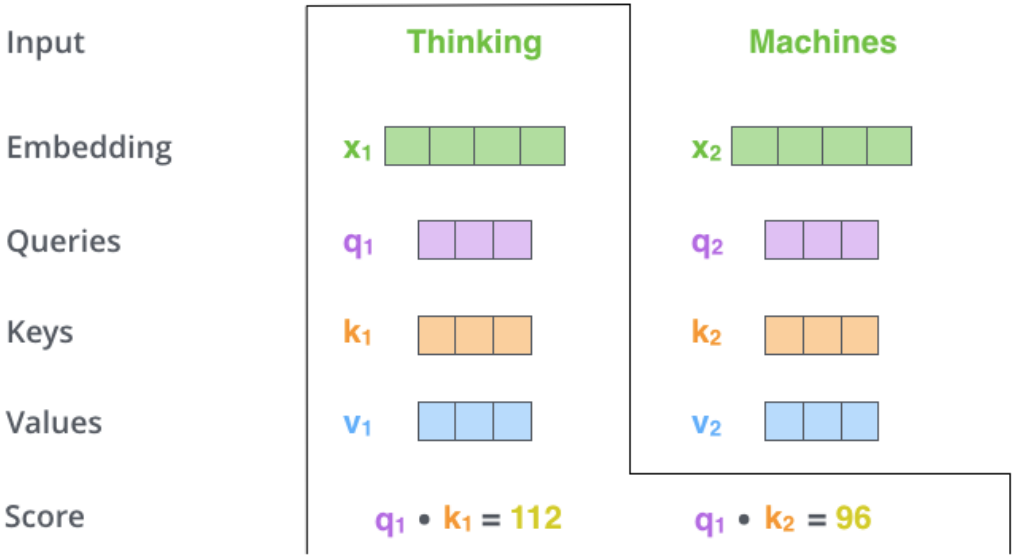



[The Illustrated Transformer – Jay Alammar – Visualizing machine learning one concept at a time.]

# Self-Attention

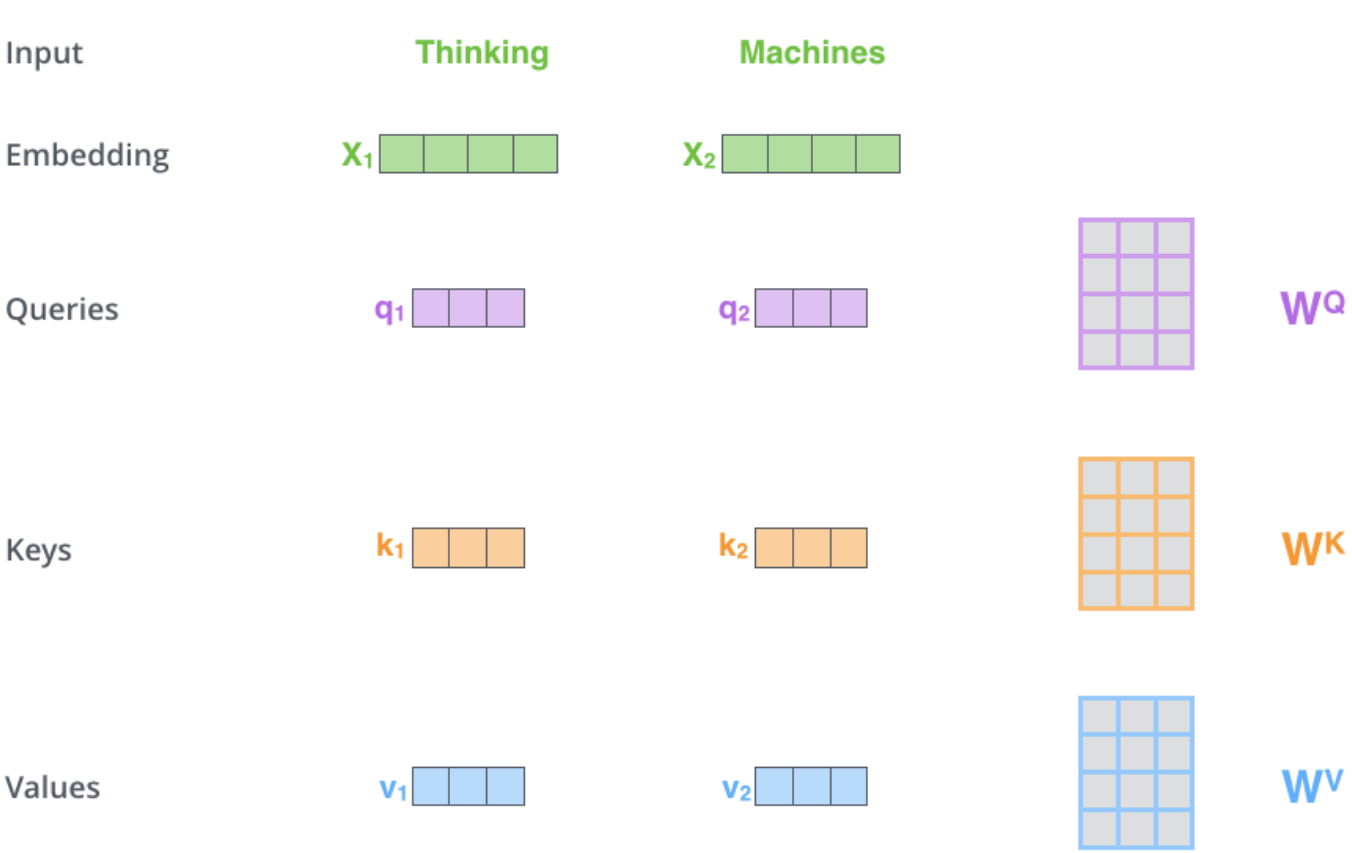
- Step 1: compute “key, value, query” embedding for each input token
- Step 2: compute scores between pairs of tokens

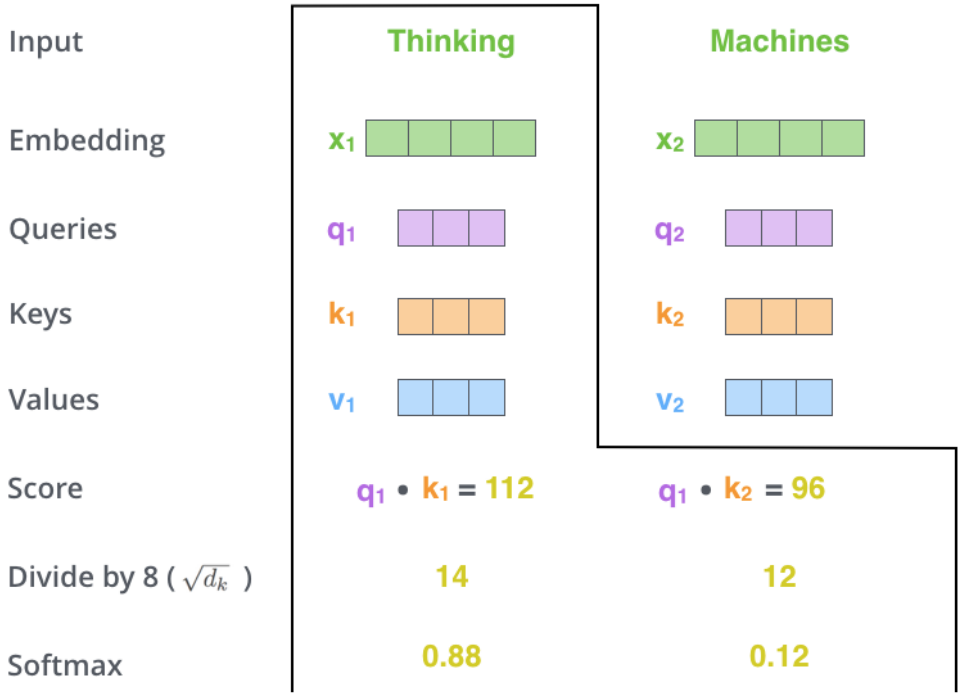


# Self-Attention

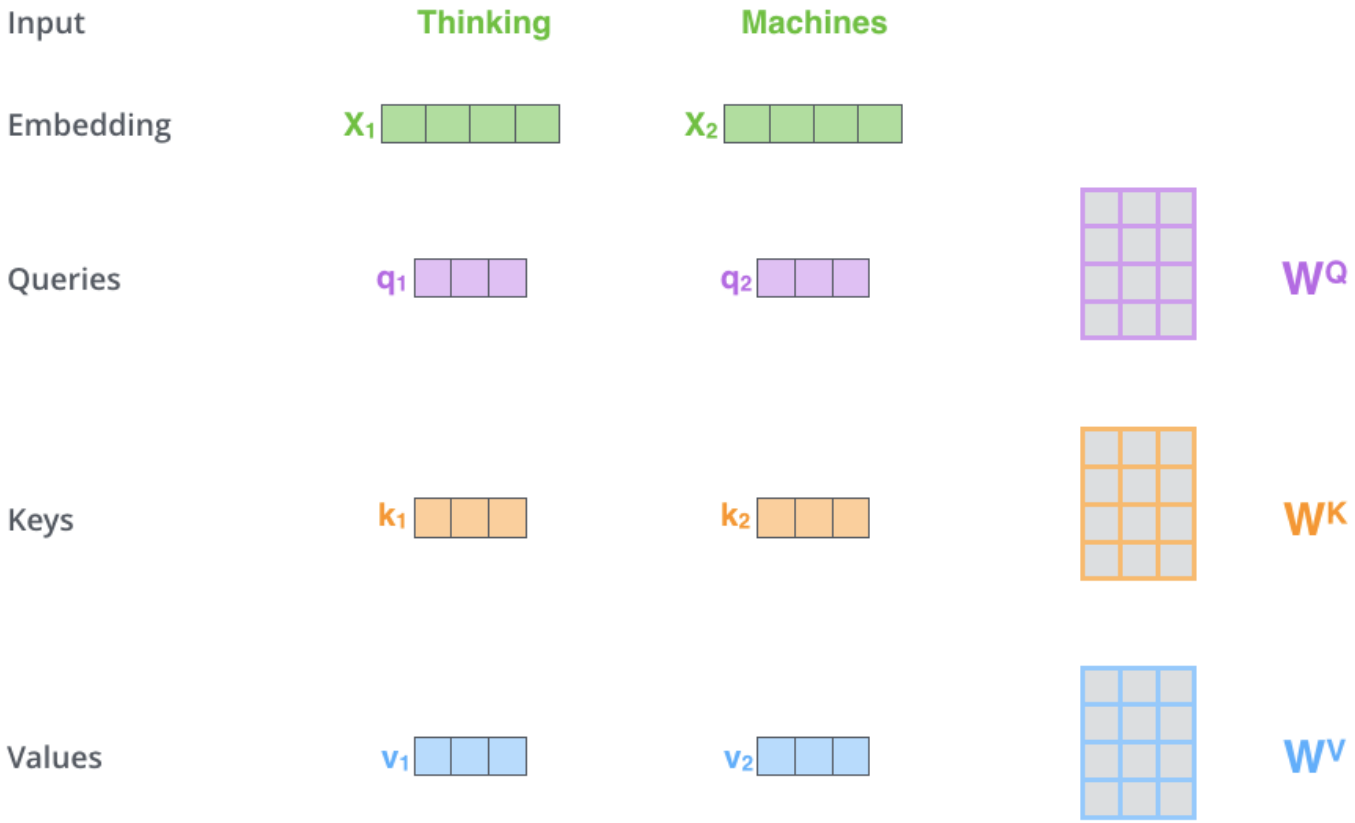
- Step 1: compute “key, value, query” embedding for each input token
- Step 2: compute scores between pairs of tokens
- Step 3: compute normalized attention scores

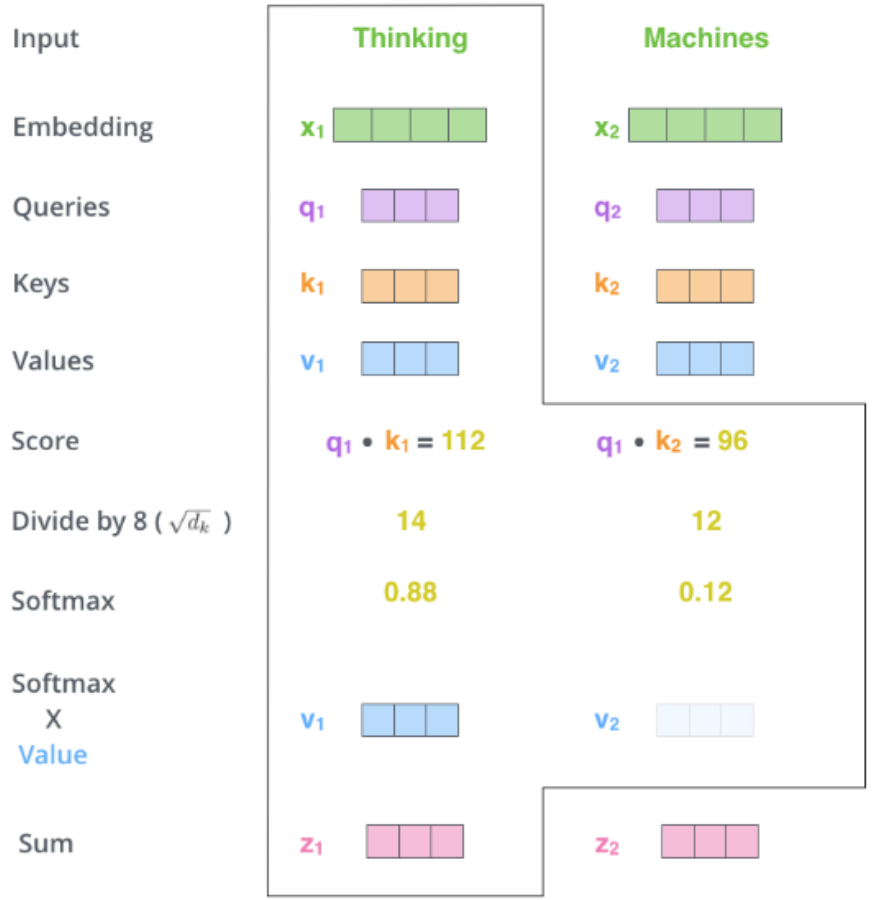


# Self-Attention

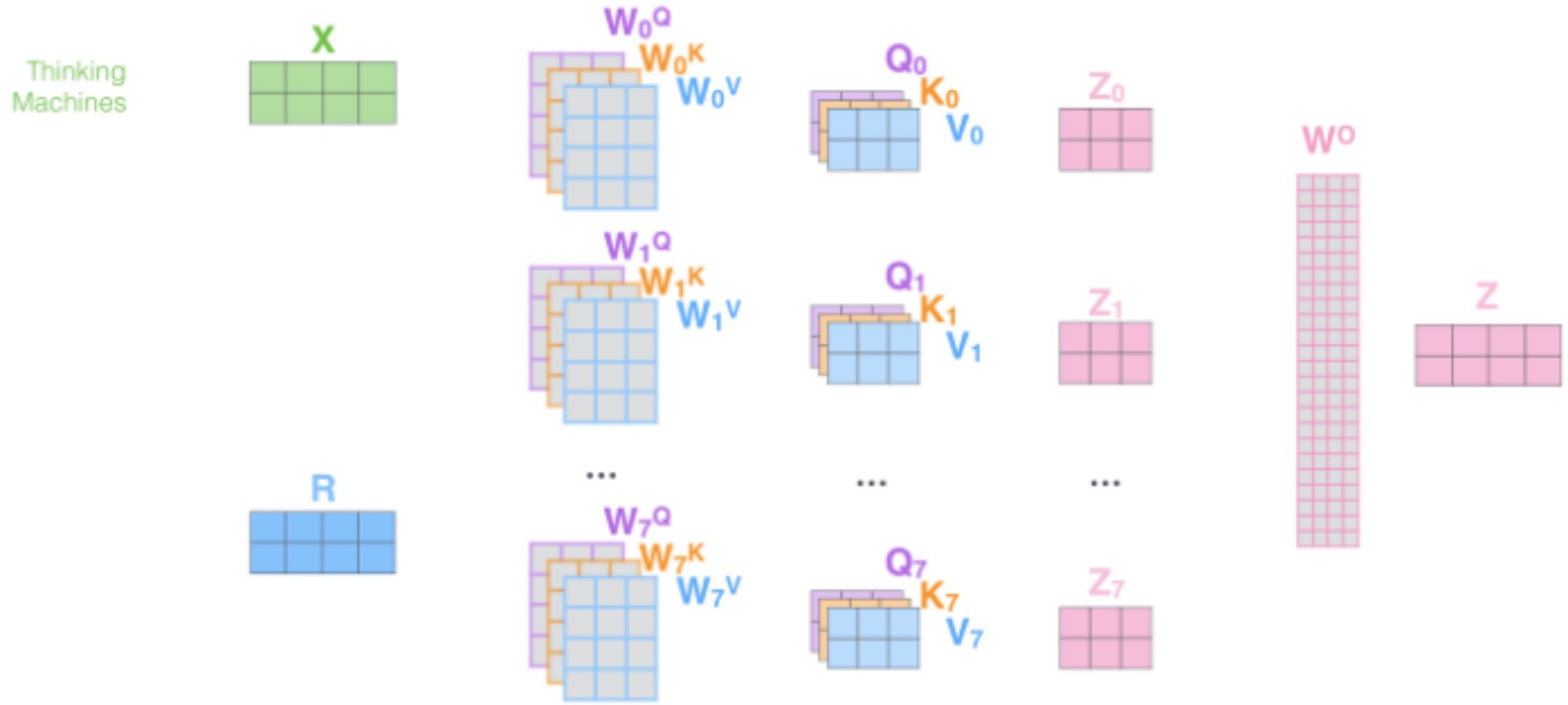
- Step 1: compute “key, value, query” embedding for each input token
- Step 2: compute scores between pairs of tokens
- Step 3: compute normalized attention scores
- Step 4: get new representation by weighted sum of values



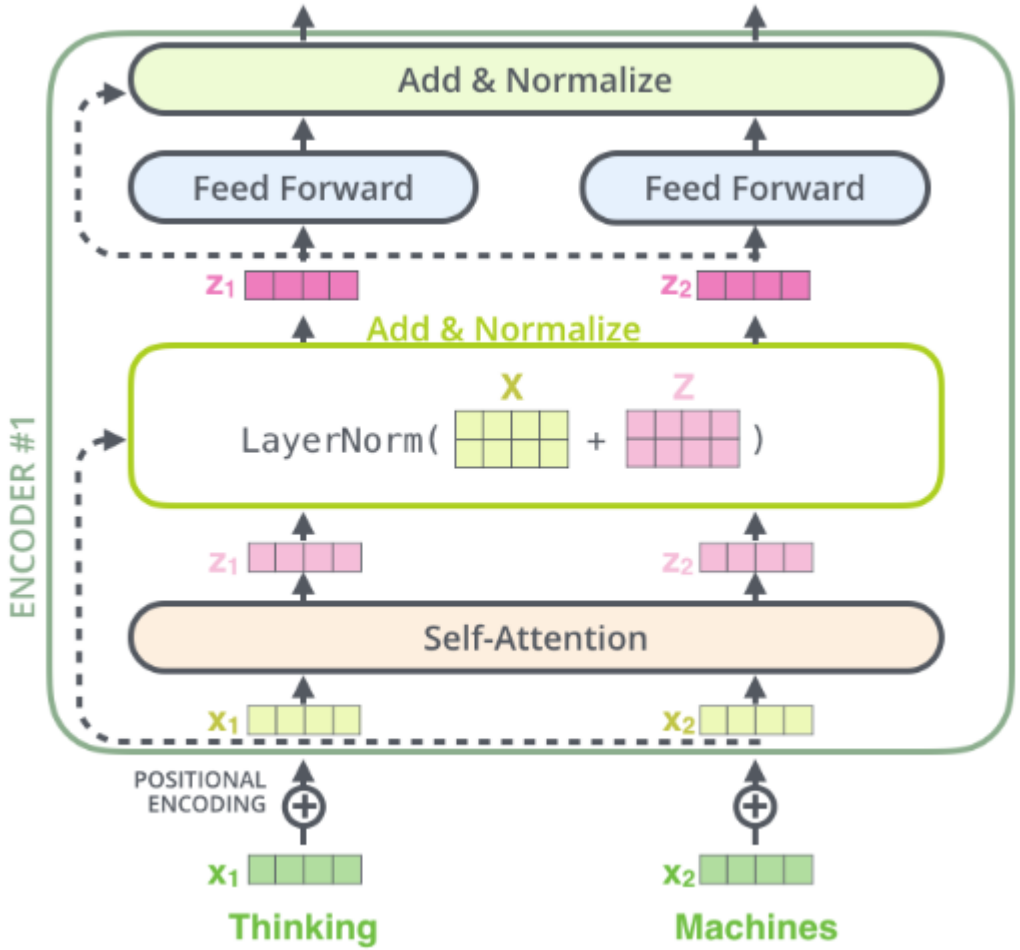



# Multi-head Attention

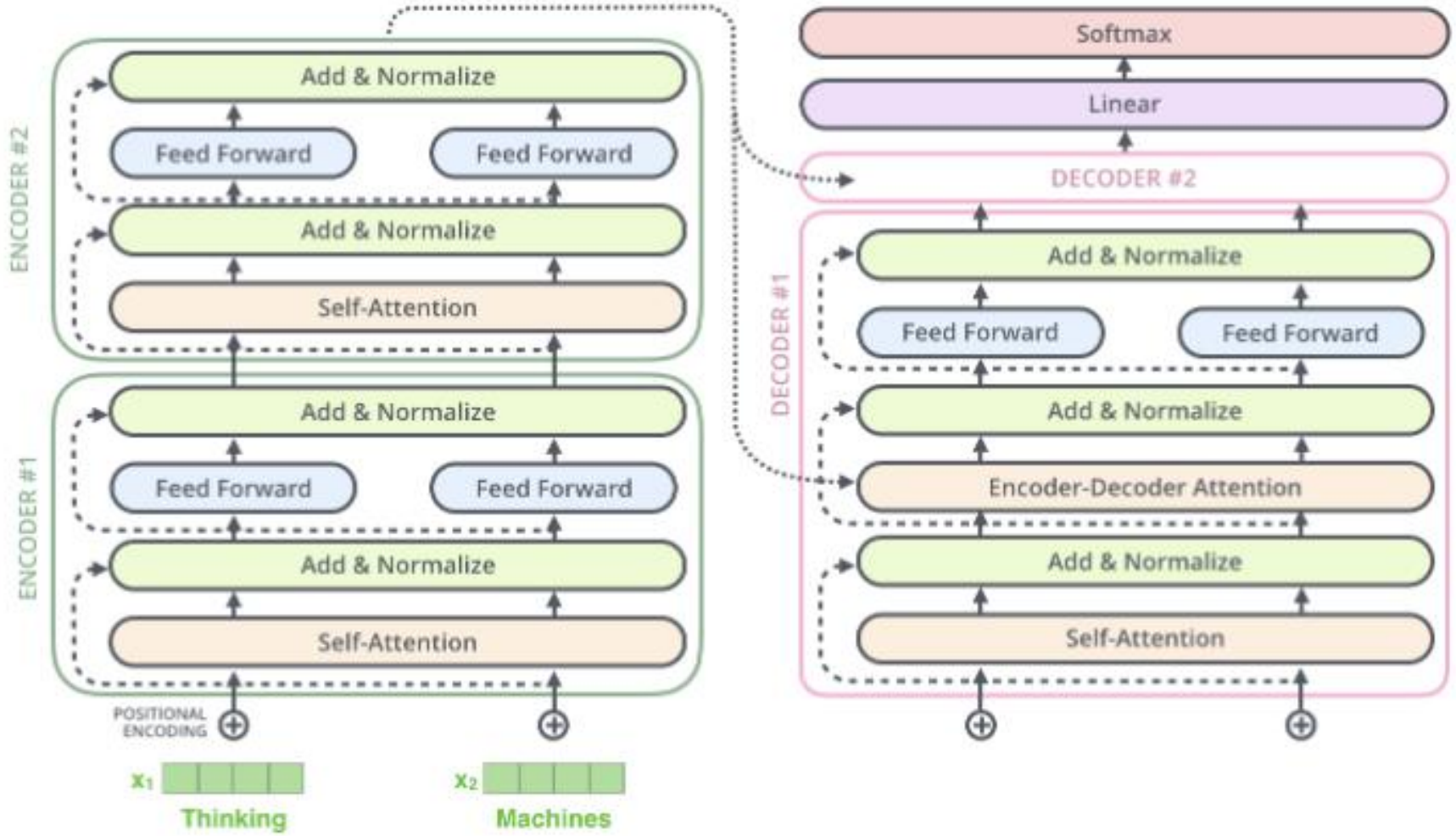
- Do many attention head calculation in parallel, and combine
- Each head has its own set of parameters
- Different heads can learn different “interactions” between inputs



# The Residuals and LayerNorm



# The Decoder Side



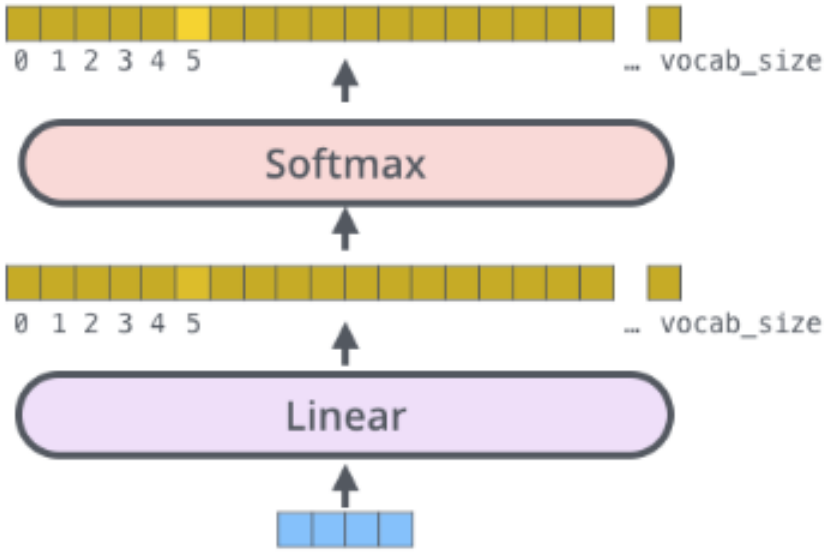
# The Final Linear and Softmax Layer

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (**argmax**)

5



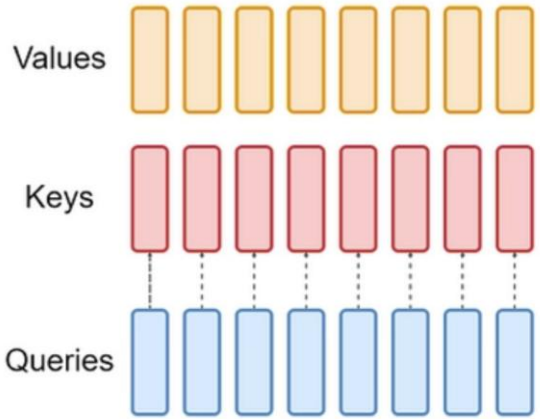
# Multi-head Attention

- Do many attention head calculation *in parallel, and combine*
- Each head has its *own* set of parameters
- Different heads can learn different “interactions” between inputs


Parallel execution is good, but *memory footprint* is still large

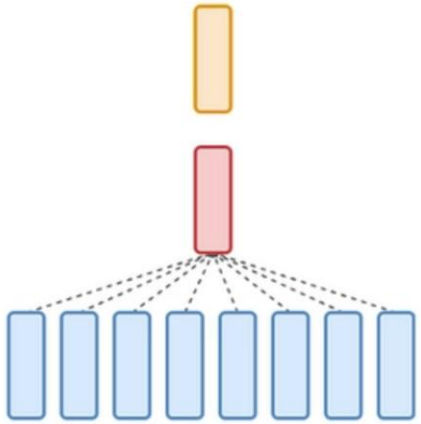
# Multi-Query Attention

***Multi-head Attention***



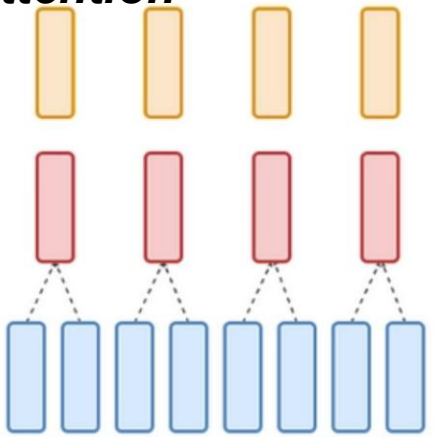

*Each head digests QKV separately*

***Multi-Query Attention***



*Share single key and value heads across query heads*

***Grouped-Query Attention***

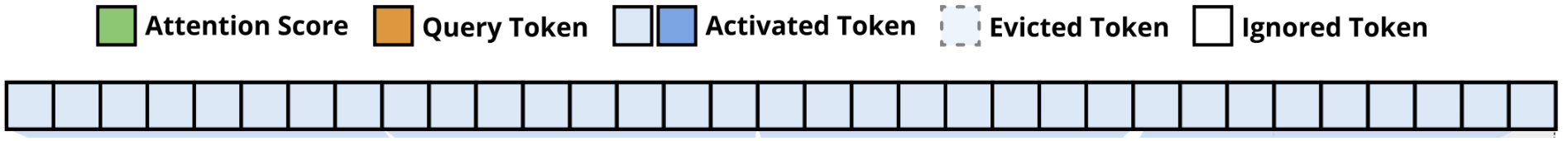


*Share single key and value heads for each group of query heads*

*Better compute, smaller memory footprint, but quality may drop*

# Native Sparse Attention (ACL'25 from DeepSeek)

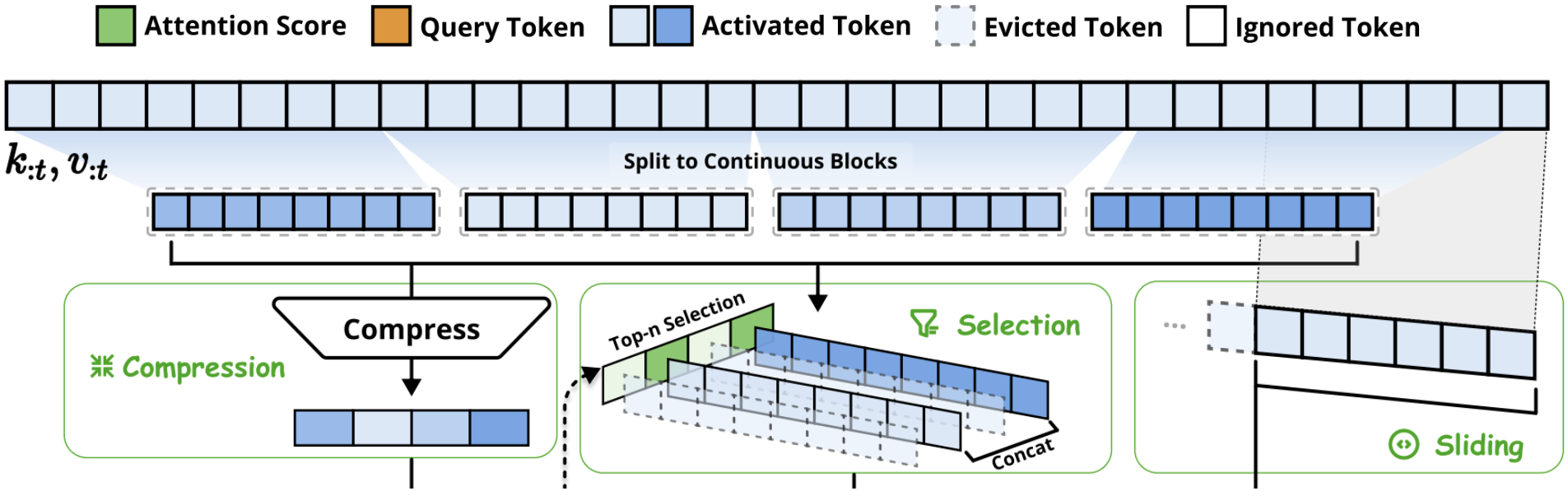
- Key insight: we can summarize long-context input, identify and focus on key words




[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

# Native Sparse Attention (ACL'25 from DeepSeek)

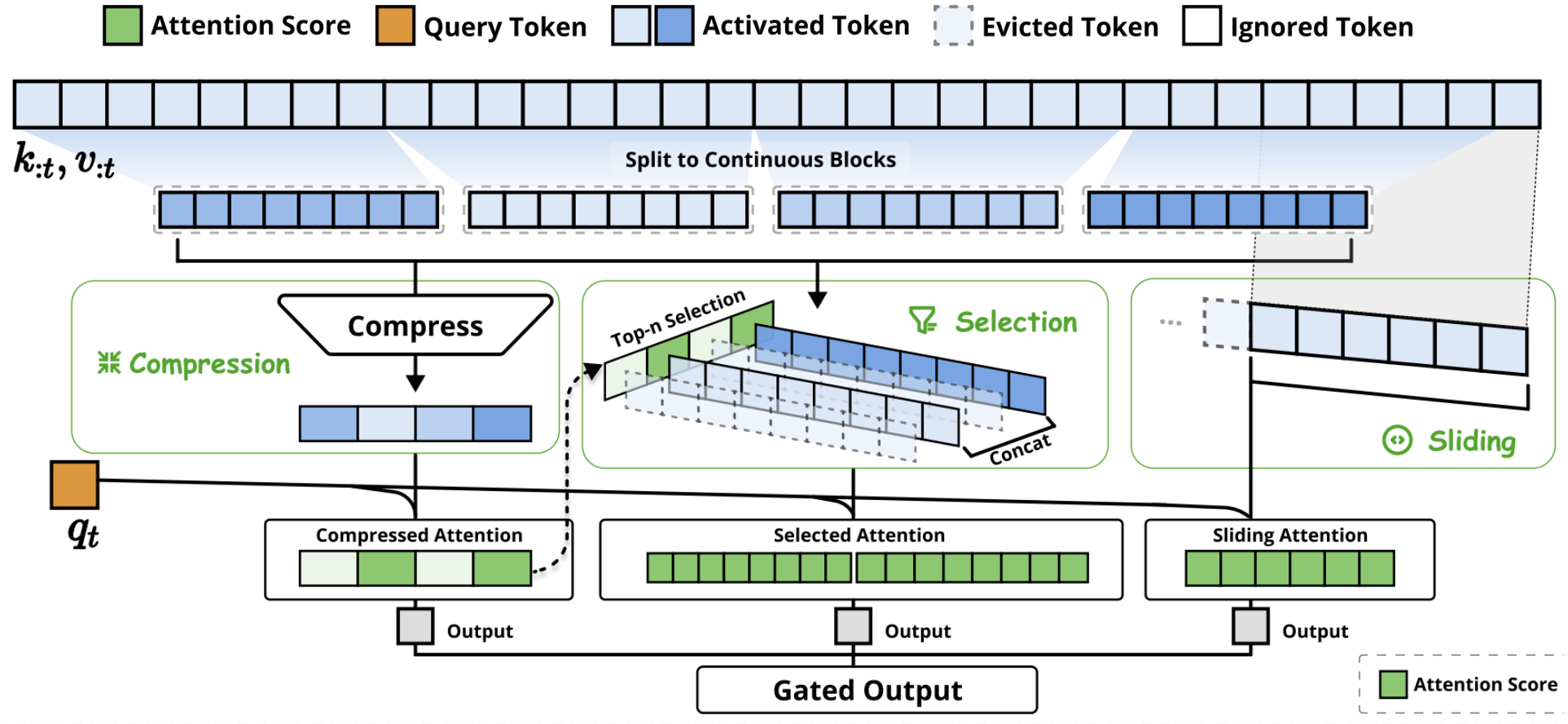
- Key insight: we can summarize long-context input, identify and focus on key words




[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

# Native Sparse Attention (ACL'25 from DeepSeek)

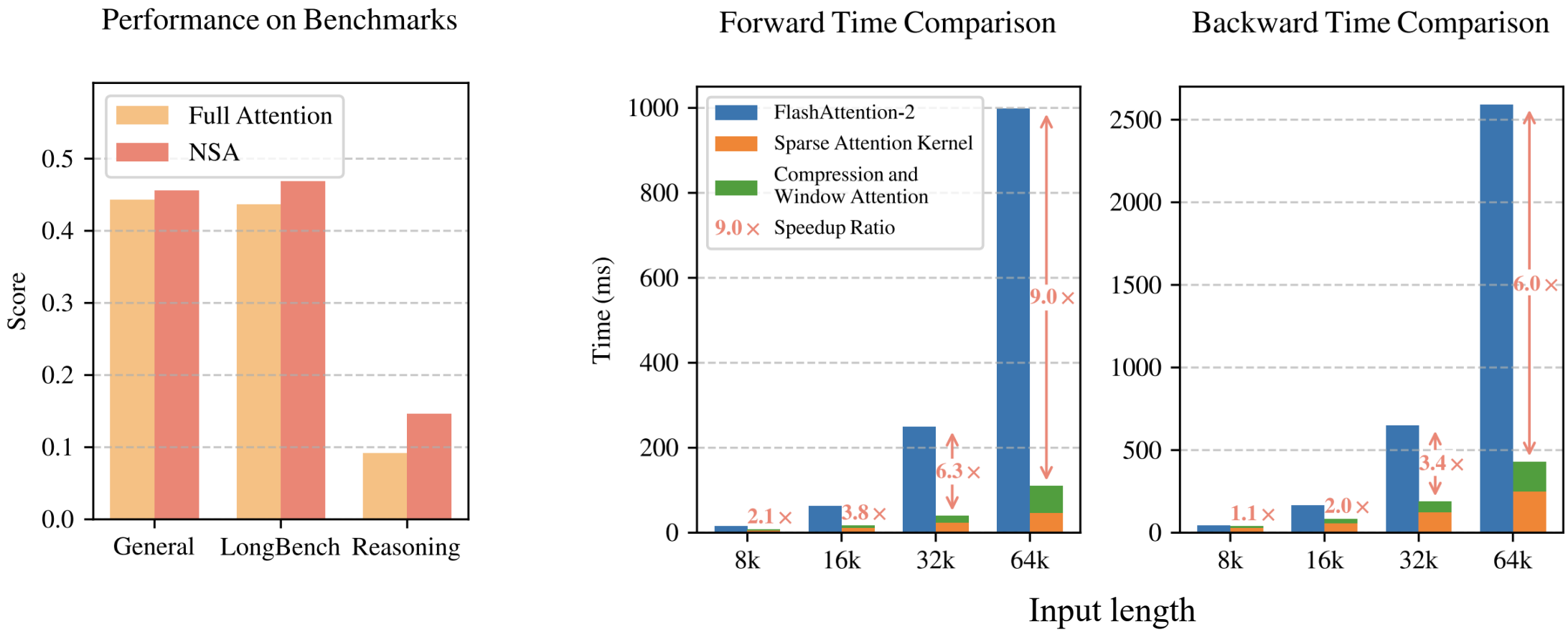
- Key insight: we can summarize long-context input, identify and focus on key words




[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

# Native Sparse Attention Performance

- Better quality, compute, and memory footprint



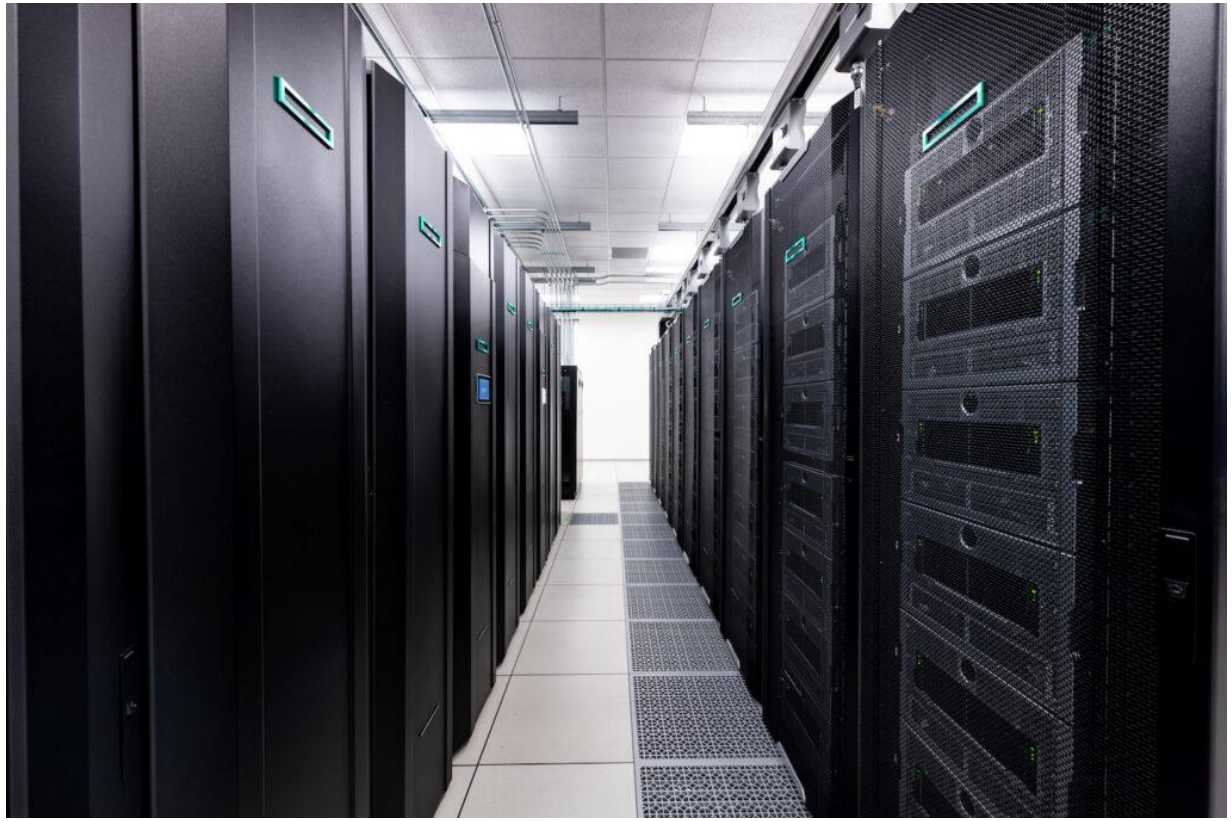

[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

# Summary

- **Transformers Deep Dive**
  - Transformer architecture
    - Tokenization, position embedding, MHSA mechanism
  - Multi-head & Multi-query Attention
    - Parallel execution of heads
  - Native Sparse Attention (ACL’25 Best Paper Award)
    - Summarize, focus on key tokens and neighboring tokens

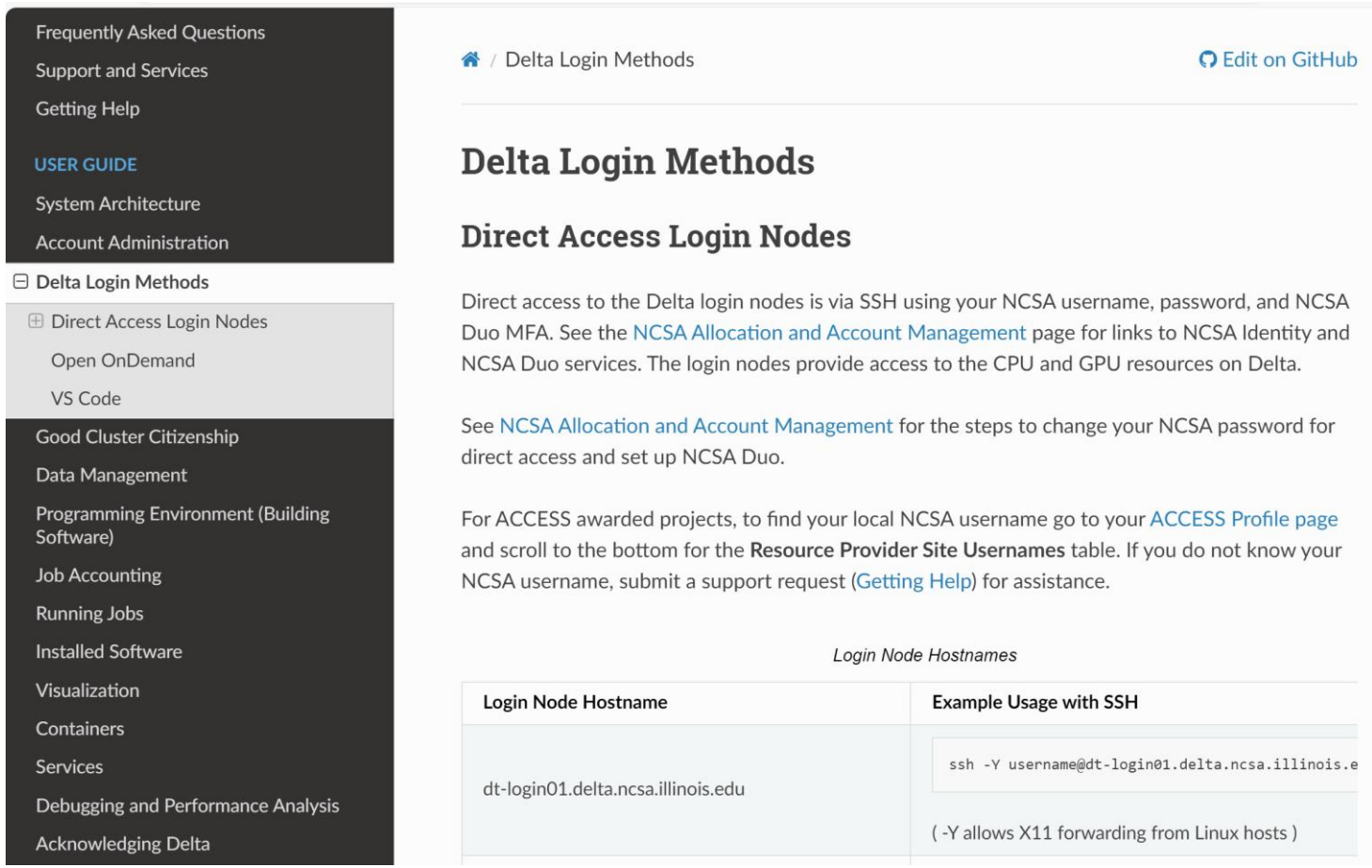
# Hardware: Delta

- Home page: https://www.ncsa.illinois.edu/research/project-highlights/delta/
  - 100 quad A100 GPU node, each with 4 A100
  - 100 quad A40 GPU node, each with 4 A40
  - 5 8-way A100 GPU, each with 8 A100
  - 1 MI100 node, 8 MI100



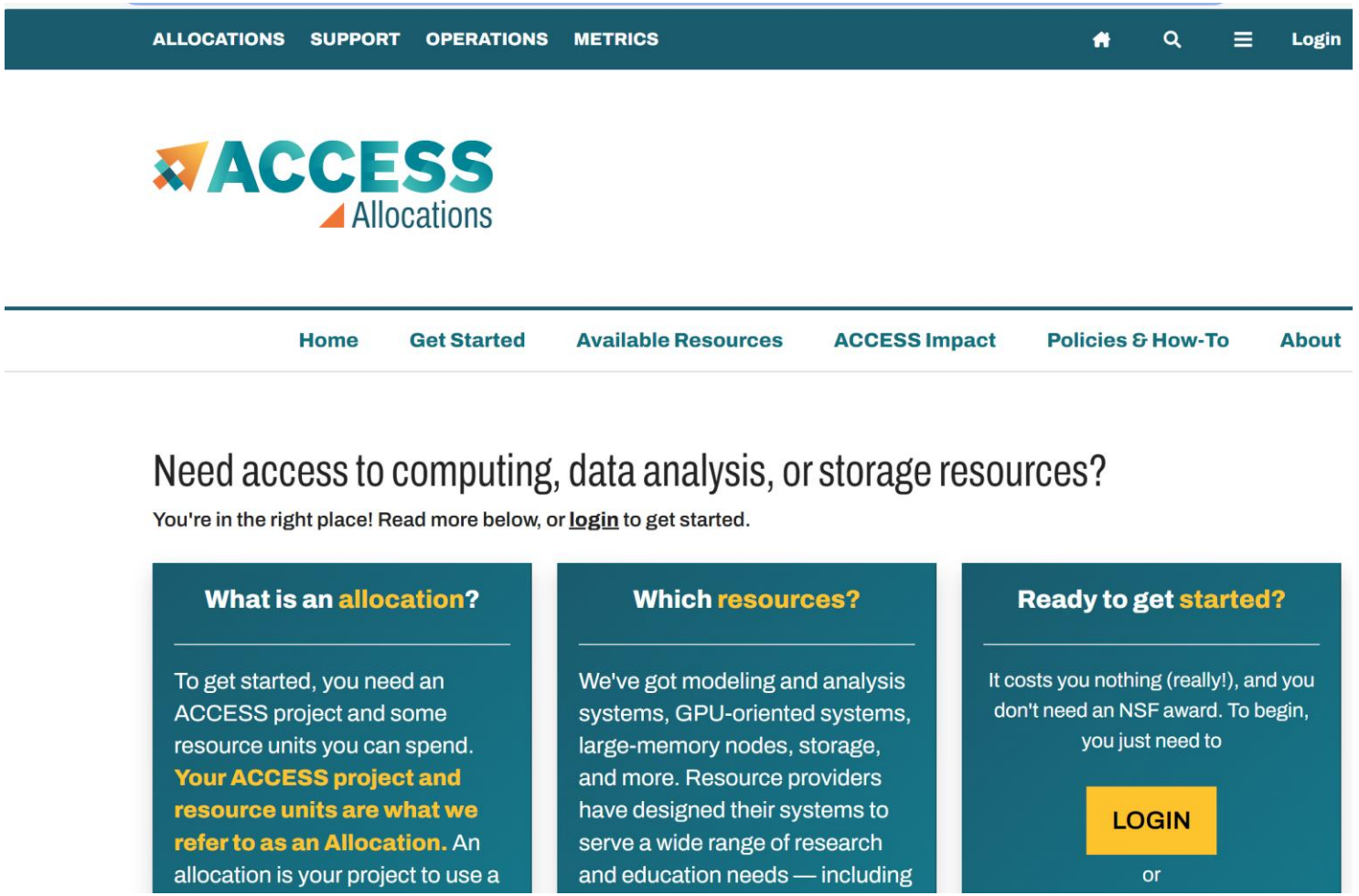
# Delta Onboarding

- https://docs.ncsa.illinois.edu/systems/delta/en/latest/user_guide/accessing.html



# Step 1: Create ACCESS ID

- Register an ACCESS id at: https://access-ci.org/ (top right-hand corner)
- After you register, send the instructor your ACCESS id. The instructor will add you to access to his GPU allocation.



# Step 2: Get Started

- Delta uses Slurm to manage jobs/GPUs
- Please watch this tutorial video: Getting Started on NCSA's Delta Supercomputer.
- After that, you may want to check Delta User Documentation — UIUC NCSA Delta User Guide (illinois.edu).
- Please learn how to use slurm to get GPUs: Slurm Workload Manager - Quick Start User Guide (schedmd.com).

# Step 3: SSH Login

- You shall use ssh to login to the node: [Delta Login Methods — UIUC NCSA Delta User Guide (illinois.edu)](Delta Login Methods — UIUC NCSA Delta User Guide (illinois.edu)).

- For instance, you can use commands such as “srun -A bcjw-delta-gpu --time=00:30:00 --nodes=1 --ntasks-per-node=16 --partition=gpuA100x4,gpuA40x4 --gpus=1 --mem=32g --pty /bin/bash”

- Maintaining Persistent Sessions: tmux

## Delta Login Methods

### Direct Access Login Nodes

Direct access to the Delta login nodes is via SSH using your NCSA username, password, and NCSA Duo MFA. See the NCSA Allocation and Account Management page for links to NCSA Identity and NCSA Duo services. The login nodes provide access to the CPU and GPU resources on Delta.

See NCSA Allocation and Account Management for the steps to change your NCSA password for direct access and set up NCSA Duo.

For ACCESS awarded projects, to find your local NCSA username go to your ACCESS Profile page and scroll to the bottom for the **Resource Provider Site Usernames** table. If you do not know your NCSA username, submit a support request (Getting Help) for assistance.

*Login Node Hostnames*

| Login Node Hostname | Example Usage with SSH |
|---|---|
| dt-login01.delta.ncsa.illinois.edu | `ssh -Y username@dt-login01.delta.ncsa.illinois.e`<br>( -Y allows X11 forwarding from Linux hosts ) |
| dt-login02.delta.ncsa.illinois.edu | `ssh -l username dt-login02.delta.ncsa.illinois.e`<br>( -l username alt. syntax for `user@host` ) |
| **login.delta.ncsa.illinois.edu**<br>(round robin DNS name for the set of login nodes) | `ssh username@login.delta.ncsa.illinois.edu` |

# Additional Info

- It is the instructor's own research allocation, and it has a limit. So please be mindful when using GPU resources.
  - Avoid allocating too many GPUs at once
  - Turn off the job when you are not using the GPUs

- The allocation has 500 GB of storage in total (shared by the class and other students in the instructor's lab)
  - Please avoid downloading large data files and super large model checkpoints, e.g., one llama7b checkpoint consumes roughly 14GB.

# Arithmetic Intensity

AI = #ops / #bytes

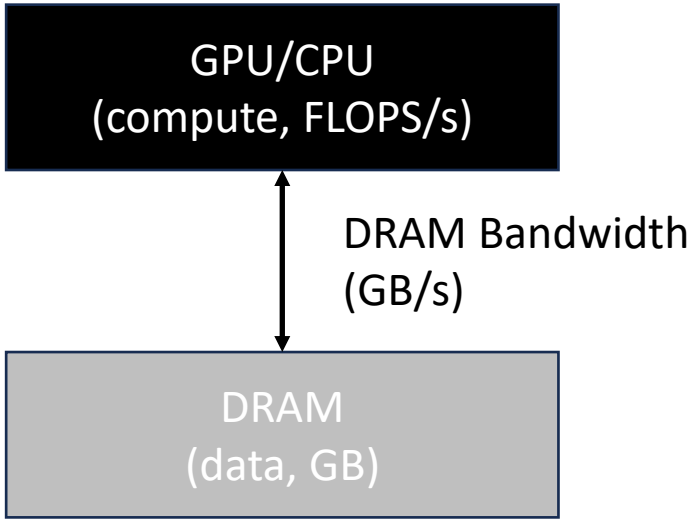
# Arithmetic Intensity

AI= #ops / #bytes

Used to evaluate the efficiency of computational algorithms

System performance is bound by

1) The peak compute TFLOPS
2) The memory bandwidth

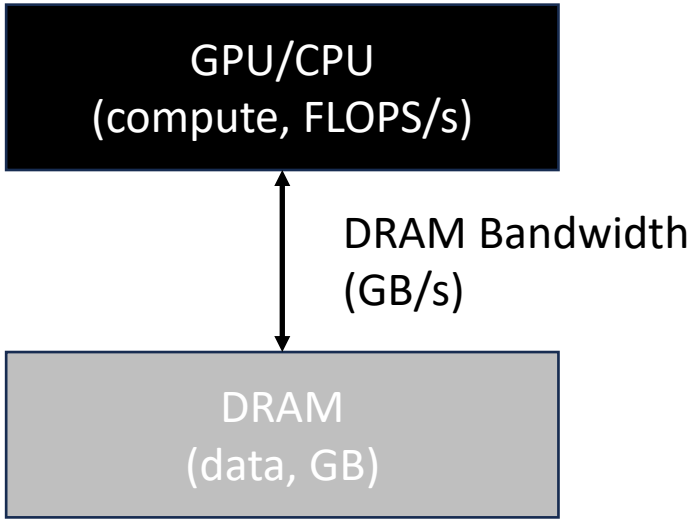


# Arithmetic Intensity

AI= #ops / #bytes

System performance is bound by

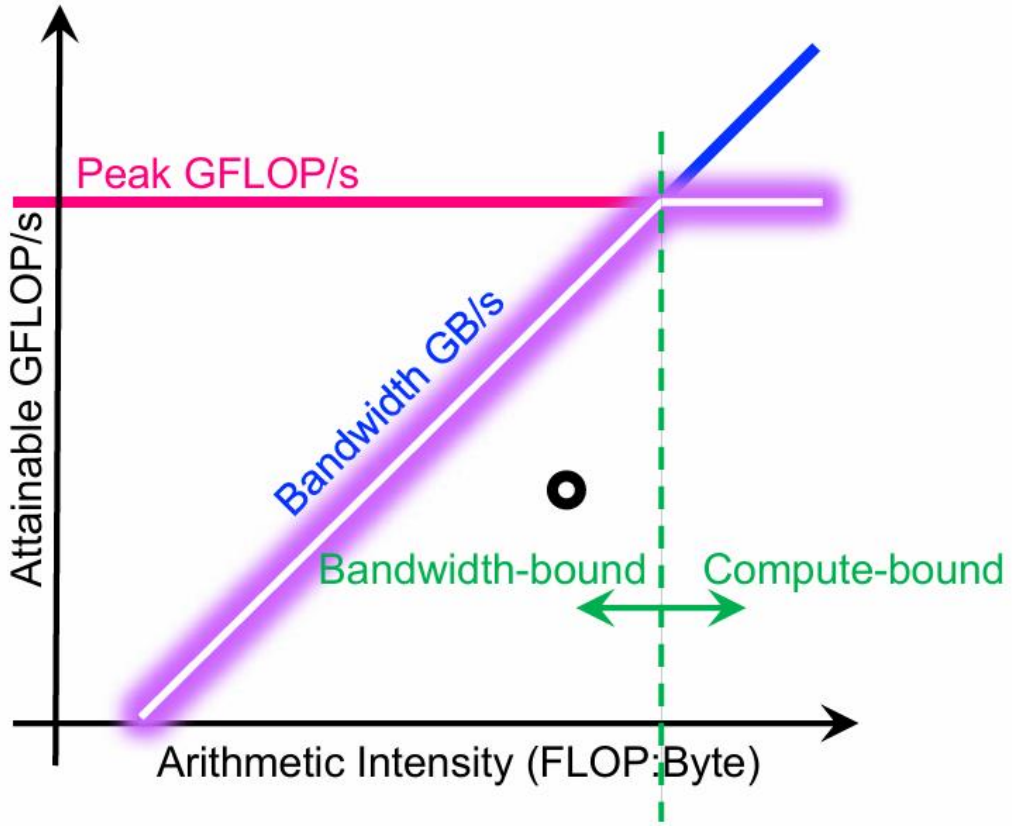
TFLOPS/s = min(Peak TFLOPS/s, Peak bw GB/s * AI)



# Roofline Model

The Roofline model provides a relatively simple way for performance estimates based on the computation of workload and hardware characteristics

- High AI: Compute-bound
- Low AI: Memory bandwidth bound




Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures, 2008

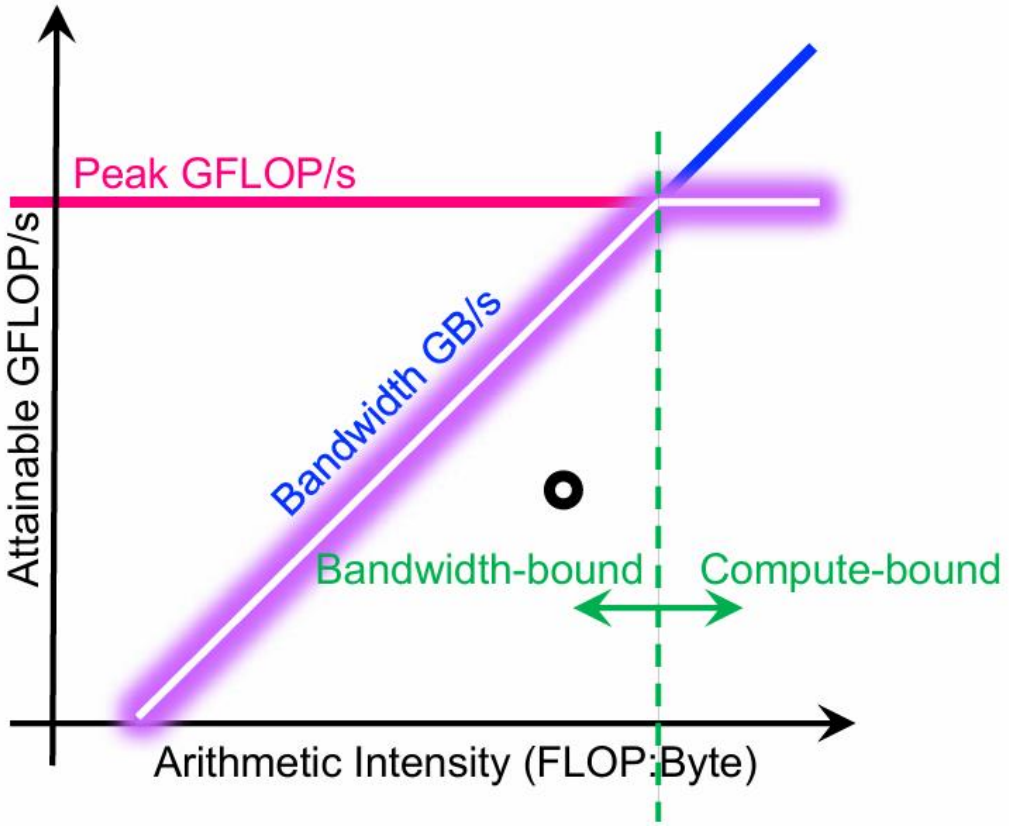
# Why Roofline Model?

Helps identify the bottlenecks

Program performance depends on how well it fits the hardware architecture

Create optimizations to exhaust both compute and bandwidth at the same time (many times it is impossible)

The mode also tells you when to stop




Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures, 2008

# Theoretical Peak of AI

AI_A100 = compute / memory_bandwidth
= 312 TFLOPS / 800 GB/s
= 390 ops/ byte

NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

| | A100 80GB PCIe | A100 80GB SXM |
|---|---|---|
| FP64 | 9.7 TFLOPS | |
| FP64 Tensor Core | 19.5 TFLOPS | |
| FP32 | 19.5 TFLOPS | |
| Tensor Float 32 (TF32) | 156 TFLOPS \| 312 TFLOPS* | |
| BFLOAT16 Tensor Core | 312 TFLOPS \| 624 TFLOPS* | |
| FP16 Tensor Core | 312 TFLOPS \| 624 TFLOPS* | |
| INT8 Tensor Core | 624 TOPS \| 1248 TOPS* | |
| GPU Memory | 80GB HBM2e | 80GB HBM2e |
| GPU Memory Bandwidth | 1,935GB/s | 2,039GB/s |
| Max Thermal Design Power (TDP) | 300W | 400W*** |
| Multi-Instance GPU | Up to 7 MIGs @ 10GB | Up to 7 MIGs @ 10GB |
| Form Factor | PCIe dual-slot air cooled or single-slot liquid cooled | SXM |
| Interconnect | NVIDIA® NVLink® Bridge for 2 GPUs: 600GB/s ** PCIe Gen4: 64GB/s | NVLink: 600GB/s PCIe Gen4: 64GB/s |

# Arithmetic Intensity

```
void add(int n, float*  A, float*  B, float*  C){
   for  (int  i=0; i<n; i++)
      C[i]  = A[i]  + B[i];
}
```

# Arithmetic Intensity

```
void add(int n, float* A, float* B, float* C){
   for (int i=0; i<n; i++)
      C[i] = A[i] + B[i];
}
```

Two loads, one store per math op (arithmetic intensity = 1/3)

1. Read A[i]
2. Read B[i]
3. Add A[i] + B[i]
4. Store C[i]

# Arithmetic Intensity

```
void add(int n, float* A, float* B, float* C){
   for (int i=0; i<n; i++)
      C[i] = A[i] + B[i];
}

void mul(int n, float* A, float* B, float* C) {
   for (int i=0; i<n; i++)
      C[i] = A[i] * B[i];
}

float* A, *B, *C, *D, *E, *tmp1, *tmp2;
//  assume arrays are allocated here
//  compute E = D + ((A + B) * C)
add(n, A, B, tmp1);
mul(n, tmp1, C, tmp2);
add(n, tmp2, D, E);
```

Two loads, one store per math op (arithmetic intensity = 1/3)

# Arithmetic Intensity

```
void add(int n, float* A, float* B, float* C){
   for (int i=0; i<n; i++)
      C[i] = A[i] + B[i];
}

void mul(int n, float* A, float* B, float* C) {
   for (int i=0; i<n; i++)
      C[i] = A[i] * B[i];
}

float* A, *B, *C, *D, *E, *tmp1, *tmp2;
//  assume arrays are allocated here
//  compute E = D + ((A + B) * C)
add(n, A, B, tmp1);
mul(n, tmp1, C, tmp2);
add(n, tmp2, D, E);
```

Two loads, one store per math op (arithmetic intensity = 1/3)

Two loads, one store per math op (arithmetic intensity = 1/3)

Overall arithmetic intensity = 1/3

# Which Program Performs Better?

```
float*  A, *B,  *C,  *D,  *E,  *tmp1, *tmp2;
//   assume arrays are allocated here
//   compute  E = D + ((A  + B) *  C)
add(n, A, B, tmp1);
mul(n,  tmp1,  C, tmp2);
add(n, tmp2,  D, E);



void fused(int  n, float*  A, float*  B, float*  C, float*  D,
   float*  E) {
   for  (int  i=0; i<n; i++)
      E[i]  = D[i]  + (A[i] + B[i]) *  C[i];
}
//   compute  E = D + (A + B) *  C
fused(n,  A, B,C,  D, E);
```

# Which Program Performs Better?

```
float*  A, *B,  *C,  *D,  *E,  *tmp1, *tmp2;
//   assume arrays are allocated here
//   compute  E = D + ((A  + B) *  C)
add(n, A, B, tmp1);
mul(n,  tmp1,  C, tmp2);
add(n, tmp2,  D, E);



void fused(int  n, float*  A, float*  B, float*  C, float*  D,
   float*  E) {
   for  (int  i=0; i<n; i++)
      E[i]  = D[i]  + (A[i] + B[i]) * C[i];
}
//   compute  E = D + (A + B) * C
fused(n,  A, B,C,  D, E);
```

Overall arithmetic intensity = 1/3

Four loads, one store per 3 math ops
Arithmetic intensity = 3/5

# Understanding Transformer Calculations

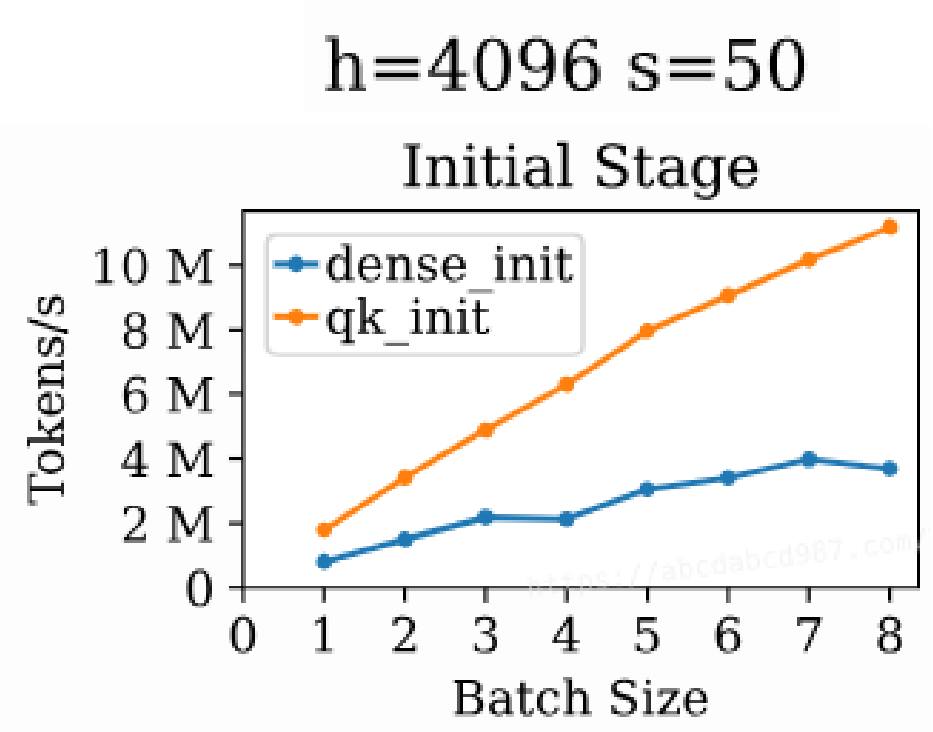
**Input** b: batch size; s: sequence length

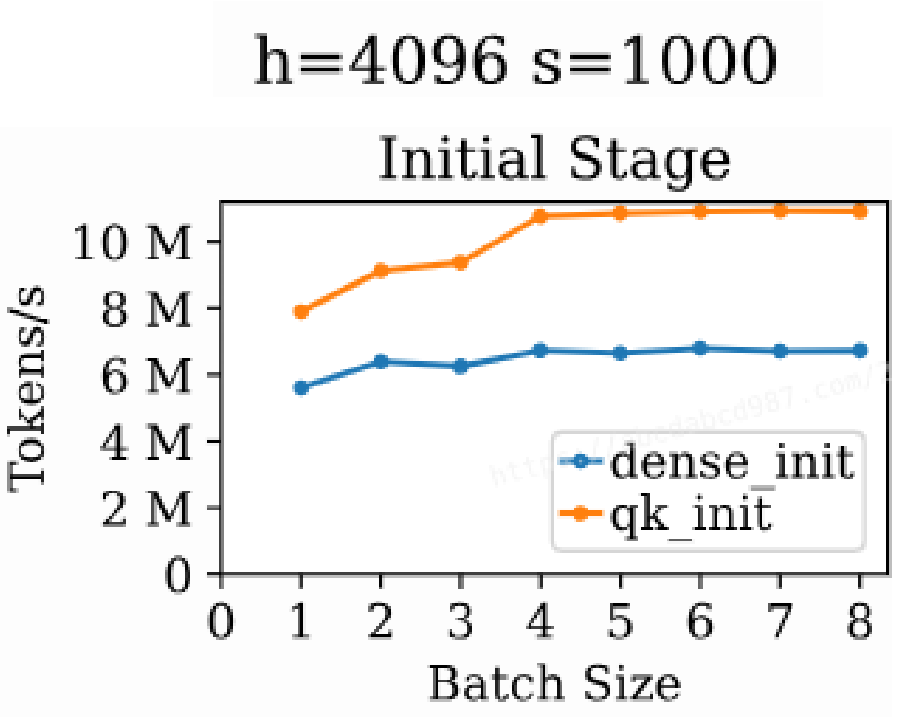
**Model** n: the number of attention heads; d: dimension of single attention head

h: hidden dimension (h = n x d)

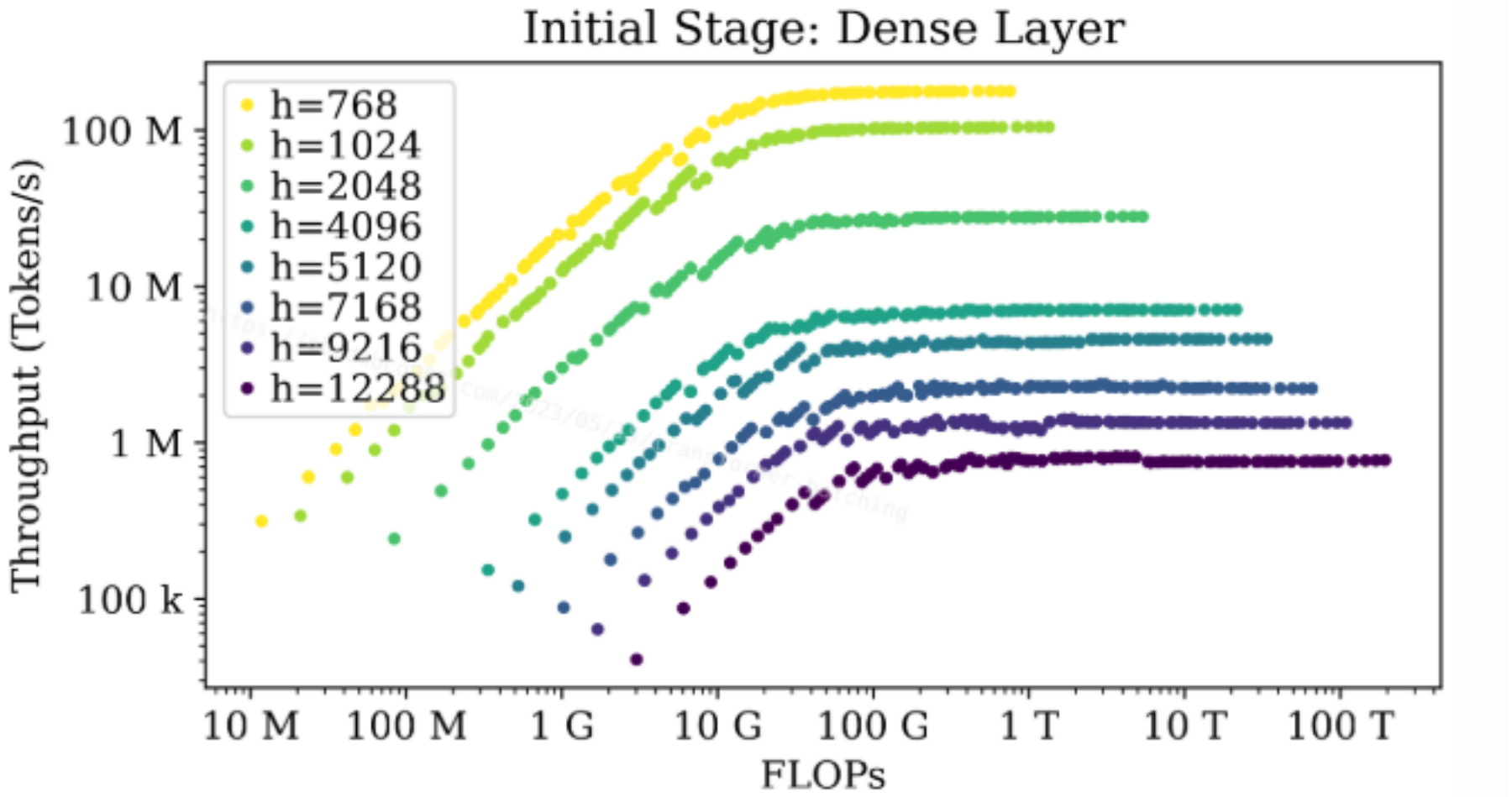
| Symbol | Definition | Shape | FLOP | I/O | FLOP:I/O |
|---|---|---|---|---|---|
| Input | | | | | |
| $X$ | Input for self attention | $(b, s, h)$ | | | |
| Self-Attention | | | | | |
| $Q^{\ddagger}, K^{\ddagger}, V^{\ddagger}$ | $XW_Q, XW_K, XW_V$ | $(b, s, h)$ | $O(bsh^2)$ | $O(2bsh + h^2)$ | $O(1/(1/h + 1/bs))$ |
| $Q^{\dagger}, K^{\dagger}, V^{\dagger}$ | Reshape $Q^{\ddagger}, K^{\ddagger}, V^{\ddagger}$ | $(b, s, n, d)$ | | | |
| $Q, K, V$ | Transpose $Q^{\dagger}, K^{\dagger}, V^{\dagger}$ | $(b, n, s, d)$ | | | |
| $K^T$ | Transpose $K$ | $(b, n, d, s)$ | | | |
| $P$ | Softmax $(QK^T/\sqrt{d})$ | $(b, n, s, s)$ | $O(bs^2nd)$ | $O(2bsnd + bs^2n)$ | $O(1/(1/d + 1/s))$ |
| $A^{\ddagger}$ | $PV$ | $(b, n, s, d)$ | $O(bs^2nd)$ | $O(2bsnd + bs^2n)$ | $O(1/(1/d + 1/s))$ |
| $A^{\dagger}$ | Transpose $A^{\ddagger}$ | $(b, s, n, d)$ | | | |
| $A$ | Reshape $A^{\dagger}$ | $(b, s, h)$ | | | |
| $Y$ | $AW_O$ | $(b, s, h)$ | $O(bsh^2)$ | $O(2bsh + h^2)$ | $O(1/(1/h + 1/bs))$ |
| Feed-Forward Network | | | | | |
| $Z$ | $\mathrm{ReLU}(YW_1)W_2$ | $(b, s, h)$ | $O(16bsh^2)$ | $O(2bsh + 8h^2)$ | $O(1/(1/h + 1/bs))$ |

# Transformer Performance: Varying Batch Sizes

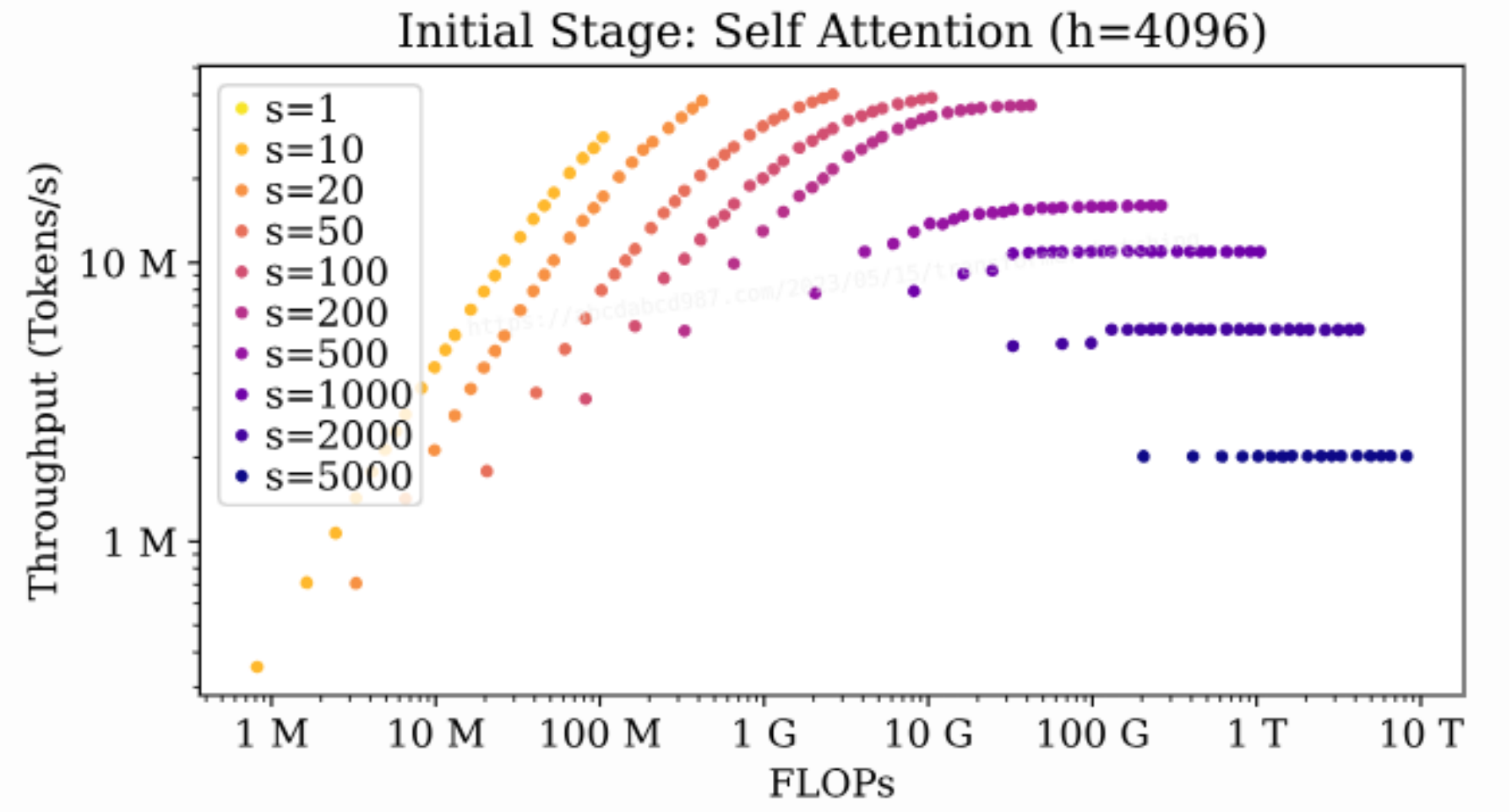


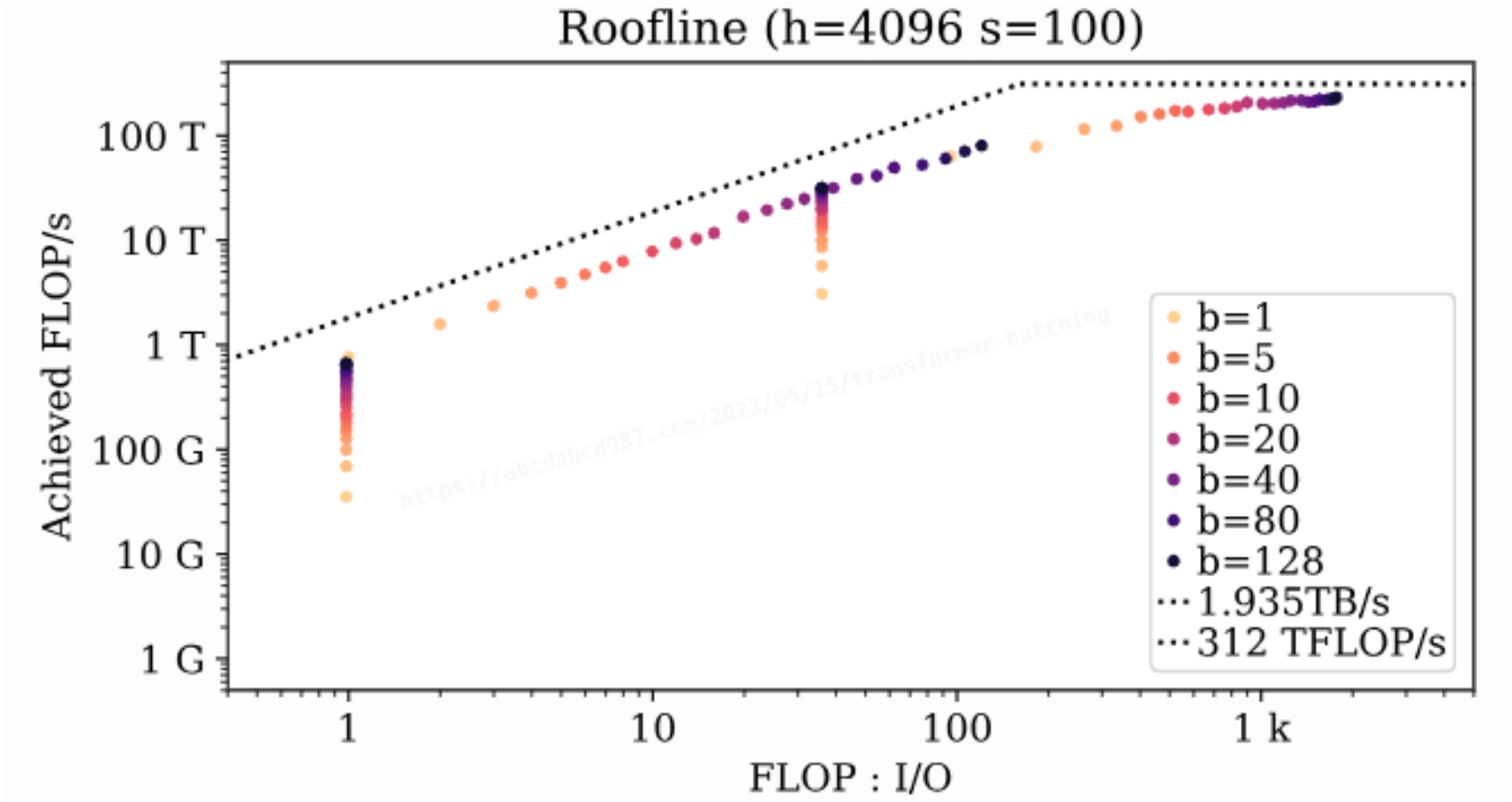
# Transformer Performance: Batching of Dense Layer



# Transformer Performance: Batching of Self-Attention



# Transformer Performance: Roofline



- After class:
  - Walk through the AI calculation of Transformers
  - How many floating-point operations in total for a 7B decoder-only model?

# Questions?